

2026 하계 연구실 로봇 플랫폼 교육 프로그램 · 플랫폼 로테이션

플랫폼 교재 — Unitree G1

휴머노이드 로봇 : 구조 이해 · DDS 통신 · MuJoCo 시뮬레이션 · 행어 모션 · 보행 미션

사용 주간 : Week 2~4 중 담당 조의 G1 로테이션 주 (W2 C조 · W3 A조 · W4 B조)

진행 : 월·화 자율학습(본 교재) → 수 퀴즈·시뮬레이션 → 금 실조작·응용 미션

담당 멘토 : G1 멘토조 (세 플랫폼 중 안전 규정이 가장 엄격한 주간입니다)

세종대학교 AI로봇학과 · 의료로봇연구실

1. G1 주간 로드맵 — 이번 주의 목표

1.1 휴머노이드는 무엇이 다른가

Go2가 "빠르게 움직이는 재미"라면, G1은 "서 있는 것 자체가 기술"인 플랫폼입니다. 사족보행은 네 다리로 안정적으로 서지만, 휴머노이드는 두 발로 균형을 잡으며 상체까지 제어해야 합니다. 연구실이 다루는 loco-manipulation(이동+조작), 전신 제어(WBC), 강화학습 sim-to-real 같은 주제들이 모두 이 플랫폼 위에 있습니다. 이번 주는 그 세계의 입구에서, 휴머노이드가 어떻게 서고 걷는지를 구조·통신·시뮬레이션·실기체 순으로 경험합니다.

1.2 주간 일정과 본 교재의 사용법

요일	활동	본 교재에서 볼 부분
월 · 화	자율학습 (교재 통독 + 예제 실행)	2~3장 정독, 4·5장 훑어보기
수 10~12시	퀴즈 · Q&A	부록 A 퀴즈 대비 체크리스트
수 14~18시	시뮬레이션 실습 (MuJoCo)	4장 (환경 준비를 화요일까지 완료)
목	자율 보충	시뮬레이션 복습 · 7장 미션 사전 설계
금 10~12시	실기체 조작 (행어 거치 · 멘토 주도)	6장
금 14~18시	응용 미션 · 정리	7장 (보행 파라미터 튜닝 + 상체 모션)

1.3 이번 주 학습 목표

- G1의 관절 구성(하체·허리·팔)과 균형 유지의 기본 원리를 설명할 수 있다
- ROS2 밑단의 DDS 통신 구조를 이해하고, G1의 상태·명령 토픽을 조사할 수 있다
- MuJoCo에서 보행을 실행하고, 파라미터가 보행에 미치는 영향을 관찰할 수 있다
- 행어 거치 상태에서 high-level API로 기립·모션 명령을 안전하게 실행할 수 있다
- 금요일 미션(보행 파라미터 튜닝 + 상체 모션 시퀀스)을 조원과 함께 완수한다

안전 재확인 — 이번 주 최우선 G1 수칙 : 행어(거치대) 미거치 상태로 단독 기립 금지 · 조작 시 멘토 포함 2인 이상, 그중 1인은 비상정지 전담 · 보행 시 반경 2m 이상 확보. 35kg의 로봇이 넘어지는 사고는 장비 파손을 넘어 사람이 다치는 사고입니다. 이번 주의 모든 실기체 절차는 멘토 주도로만 진행됩니다.

2. G1 훑어보기 — 구조와 균형의 원리

이번 장의 목표는 두 가지입니다. 첫째, G1이 어떤 관절들로 이루어져 있고 각 부위가 무슨 역할을 하는지 — "하체는 균형, 상체는 작업"이라는 지도를 그릴 수 있게 되는 것. 둘째, 휴머노이드가 왜 넘어지는지, 그리고 그것이 왜 이번 주의 엄격한 안전 절차(행어 거치·멘토 주도)로 직결되는지를 원리 수준에서 이해하는 것입니다. 이 장을 제대로 읽으면 금요일에 행어 위의 G1을 보는 눈이 완전히 달라집니다 — 수요일 퀴즈의 절반이 여기서 나옵니다.

2.1 관절 구성 — 사람의 몸을 본뜬 기계

G1은 사람의 신체 구조를 본떠 수십 개의 관절(자유도)을 갖습니다. 크게 세 부분으로 나누어 이해합니다.

부위	구성과 역할
하체 (다리 × 2)	다리당 6자유도(고관절 3 + 무릎 1 + 발목 2) — 보행과 균형의 주역
허리 (torso)	상체 회전·기울임 — 이동 중 균형 보상과 작업 공간 확장
팔 (× 2)	팔당 다관절 구성 — 제스처·조작(manipulation)에 사용, 이번 주 상체 모션의 대상
IMU · 내장 컴퓨터	몸통 기울기·각속도 측정과 실시간 균형 제어 연산

정확한 자유도 수는 모델 구성(손 장착 여부 등)에 따라 다르므로, 우리 연구실 기체의 사양은 멘토가 첫 시간에 안내합니다. 중요한 것은 숫자보다 "하체는 균형, 상체는 작업"이라는 역할 구분입니다.

다리 하나를 해부하면 — 6자유도의 의미

Go2의 다리가 3자유도였다면 G1의 다리는 6자유도입니다. 자기 다리로 하나씩 확인하며 읽어 보세요 — 여러분의 다리가 정확히 이 구성입니다.

- **고관절 3 축 (roll · pitch · yaw) :** 다리를 옆으로 벌리고(roll), 앞뒤로 흔들고(pitch), 안팎으로 돌리는(yaw) 세 회전 — 발을 "어디에" 디딜지 정하는 축들
- **무릎 1 축 :** 다리를 접었다 펴는 축 — 보폭과 지면 높이 적응
- **발목 2 축 (pitch · roll) :** 발끝을 들거나(pitch) 발바닥을 좌우로 기울이는(roll) 축 — 지면에 닿은 발바닥 안에서 힘의 중심을 미세 조정하는, 균형의 최전선

Go2와의 차이가 여기서 드러납니다. 사족은 다리 끝이 "점"으로 지면을 딛기 때문에 발목이 필요 없지만, 휴머노이드는 "발바닥"이라는 면으로 서기 때문에 그 면을 다루는 발목 관절이 필수입니다. 발목 2자유도가 바로 다음 절의 균형 이야기와 연결됩니다.

팔은 왜 다관절인가 — 이번 주 상체 모션의 배경

팔의 관절이 많은 이유는 손끝을 "원하는 위치에, 원하는 방향으로" 보내기 위해서입니다. 공간에서 위치 3 + 방향 3을 자유롭게 정하려면 최소 6자유도가 필요하다는 것이 로봇 팔의 기본 원리이며(다음 로테이션의 FR3가 7자유도인 이유이기도 합니다), G1의 팔도 이 목적으로 설계되어 있습니다. 이번 주에는 팔로 물건을 조작하지는 않고, 정해진 모션(제스처·시퀀스)을 안전하게 실행하는 것까지를 다룹니다 — 7장 미션의 "상체 모션 시퀀스"가 그 무대이고, 조작의 본격 세계는 FR3 주간과 이후 연구에서 이어집니다.

관찰 포인트 연구실에서 행어에 거치된 G1을 눈으로만 관찰해 보세요(전원 여부와 무관하게 접촉 금지 — 실기체 접근은 금요일 멘토 주도 시에만). 고관절 세 축이 어디서 교차하는지, 발목이 어떻게 두 방향으로 꺾이는지, 허리와 팔의 관절 위치를 실물로 확인하면 이 절이 한 번에 정리됩니다.

2.2 휴머노이드는 왜 넘어지는가 — 균형의 직관

두 발로 선 로봇은 발바닥이 만드는 좁은 지지 영역(support polygon) 위에 무게중심을 계속 유지해야 합니다. 무게중심이 이 영역을 벗어나는 순간 로봇은 넘어지기 시작하고, 제어기는 발을 옮겨 딛거나(스텝) 상체를 움직여(자세 보상) 이를 막습니다. 걷기란 "계획적으로 넘어지며 발을 받쳐 넣는 일"이라는 표현이 있을 정도로, 보행 중 균형 제어는 매 순간 수행되는 고속 연산입니다.

[위에서 본 지지 영역]

두 발 지지 :  두 발바닥 + 그 사이 = 넓다 → 안정
한 발 지지 :  발바닥 하나 = 좁다 → 보행의 매 순간
(걷기 = 넓은 영역과 좁은 영역을 초당 수 회 오가는 일)

균형을 지키는 두 가지 무기

- **자세 보상 (작은 흔들림)** : 발목으로 발바닥 안의 힘 중심을 옮기고, 허리·팔로 상체를 기울여 무게중심을 되돌립니다 — 2.1 절의 발목 2 축과 허리가 쓰이는 곳
- **스텝 (큰 흔들림)** : 자세 보상으로 부족하면 발을 옮겨 지지 영역 자체를 무게중심 아래로 다시 가져옵니다 — 사람이 밀렸을 때 반사적으로 발이 나가는 것과 같은 원리

이 직관이 안전 규정과 직결됩니다: 제어가 꺼지거나 오작동하면 균형 연산이 멈추고, 로봇은 그대로 무너집니다 — 그래서 행어 거치가 기본입니다. 또한 보행 파라미터(보폭·주기·발 높이 등)를 바꾸면 균형 여유가 달라집니다 — 7장 미션에서 직접 관찰합니다.

무게중심이 높다는 것 — 안전거리 2 m의 근거

Go2와 비교해 G1의 균형이 근본적으로 어려운 이유가 하나 더 있습니다 — 무게중심의 높이입니다. 사족은 무게중심이 낮고 지지 영역이 넓지만, 휴머노이드는 사람처럼 무게중심이 높은 곳에 있어 같은 크기의 기울어짐도 훨씬 빠르게 커집니다(긴 막대를 손바닥에 세우는 것과 짧은 막대의 차이를 떠올려 보세요 — 단, 긴 쪽이 "천천히 넘어지는" 대신 일단 기울면 되돌리기 어렵습니다). 그리고 35kg의 질량이 그 높이에서 넘어지며 만드는 운동은 사람에게 실제 위험입니다.

여기서 이번 주 안전 수칙의 숫자들이 나옵니다 — 보행 시 반경 2 m는 "로봇의 키 + 넘어지며 뺨는 팔다리 + 여유"를 고려한 최소 거리이고, 비상정지 전담 1인은 "균형이 무너지기 시작한 뒤에는 사람이 개입할 시간이 거의 없다"는 사실의 반영입니다. 규칙을 외우는 것과 규칙의 물리를 이해하는 것은 다릅니다 — 후자가 이 절의 목표입니다.

사람으로 하는 실험은 사람에게만 균형 직관을 몸으로 확인하는 가장 좋은 실험은 자기 몸입니다 (미션 ②). 로봇을 밀어보는 외란 실험은 절대 금지 — 궁금하면 4장의 MuJoCo 시뮬레이션에서 안전하게 해 봅니다.

2.3 Go2와 무엇이 다른가 — 지난 주와의 연결

이미 Go2를 경험한 조라면(또는 곧 경험할 조라면) 아래 대비가 이번 주의 지도가 됩니다. 명령 철학은 같고, 균형의 난이도와 안전 절차가 다릅니다.

관점	Go2 (사족)	G1 (휴머노이드)
정적 안정성	네 다리 — 서 있는 것이 기본	두 발 — 서 있는 것도 제어의 결과
속도 명령	Move(vx, vy, vyaw)	동일한 3요소, 더 보수적인 값
시뮬레이터	Gazebo 기반	MuJoCo (접촉 동역학 강점)
실기체 기본자세	지정 구역 내 자유 주행	행어 거치가 기본, 보행은 멘토 주도
상체	없음	허리·팔 — 모션·조작의 무대

같은 것 셋, 새로운 것 셋

변하지 않는 것 — ① 탐사 루틴(topic list부터), ② "시뮬레이션 검증 → 실기체" 사이클, ③ "의도를 말하면 균형은 로봇이 책임"이라는 high-level 철학. 새로 만나는 것 — ① 두 발 균형이라는 근본 난이도, ② 상체(허리·팔)라는 새 무대, ③ MuJoCo라는 새 시뮬레이터(왜 바뀌는지는 4.1절에서 — 발바닥과 지면의 접촉을 정밀하게 다루는 것이 휴머노이드 시뮬레이션의 핵심이기 때문입니다). Go2 주간에 만든 습관 위에 세 가지만 얹으면 됩니다.

1장의 용어 미리 풀기 — 연구실 연구와의 연결

1.1절에 등장한 연구 용어 세 개를 이 장의 언어로 미리 풀어 둡니다. "내가 지금 배우는 것이 어느 연구로 이어지는가"의 지도가 됩니다.

용어	이 장의 언어로 풀면
loco-manipulation	이동(loco)과 조작(manipulation)을 동시에 — "하체는 균형, 상체는 작업"을 한 몸에서 함께 푸는 문제
전신 제어 (WBC)	수십 개 관절을 부위별이 아니라 몸 전체로 함께 조율 — 균형과 작업 목표를 한꺼번에 만족시키는 계산

용어	이 장의 언어로 풀면
강화학습 sim-to-real	시뮬레이션에서 보행 정책을 학습해 실기체로 옮기는 접근 — 4장의 MuJoCo가 바로 그 학습 무대로 널리 쓰입니다

2.4 제어 계층 — 이번 주 우리는 어디에 명령하는가

계층	내용	이번 주
low-level 제어	각 관절의 토크·각도를 직접 명령. WBC·RL 제어기를 직접 엮을 때 사용	사용 안 함 (연구 단계)
high-level API	기립·보행·모션 등 동작 단위 명령 — 내장 제어기가 균형·관절을 책임	이번 주의 대상
리모컨	수동 조종·비상정지	실기체 안전 장치

개념 연결 Go2의 sport mode와 같은 철학입니다 — "의도를 말하면 균형은 로봇이 책임진다". 다만 휴머노이드는 균형의 난이도가 훨씬 높아, 같은 high-level 명령이라도 실행 전 확인 절차가 더 엄격합니다.

상태 기계 — 순서를 건너뛸 수 없다

휴머노이드의 high-level 제어에서 특히 중요한 개념이 상태(state)와 그 전이입니다. G1은 "엎드림/거치 → 준비 자세 → 기립(균형) → 보행" 같은 상태들을 정해진 순서로만 오갈 수 있습니다 — 엎드린 로봇에 보행 명령을 보내는 것은 무의미하거나 위험하며, 좋은 시스템은 이런 잘못된 전이를 거부합니다. 5장의 API 실습에서 "현재 상태 확인 → 다음 상태로 전이 명령 → 전이 완료 대기"라는 리듬을 계속 만나게 되는 이유입니다. Go2의 "StandUp 후 sleep"이 여기서 한층 더 엄격한 규칙이 됩니다.

행어와 함께 일하는 법

행어(거치대)는 단순한 보관 장치가 아니라 이번 주 실습의 안전 파트너입니다. 로봇의 몸통을 위에서 잡아 주므로 — 균형 제어가 실패하거나 상태 전이가 잘못되어도 로봇이 바닥까지 떨어지지 않습니다. 행어 거치 상태에서도 할 수 있는 것이 많습니다: 기립 자세 잡기, 균형 제어 동작 관찰, 팔·허리 모션 실행(금요일 오전의 주 내용, 6장). 반대로 보행은 발이 체중을 싣고 지면을 딛어야 의미가 있으므로, 행어에서 내려 진행하는 절차 전체를 멘토가 주도합니다. "행어에서 무엇까지 하고, 언제 내리는가"의 판단은 전적으로 멘토의 몫 — 여러분은 요청하지 않습니다.

사람의 손이 항상 우선한다

Go2 주간과 동일한 원칙이 더 강하게 적용됩니다 — 모든 실기체 절차에서 멘토가 리모컨(비상정지)을 들고, 여러분의 코드보다 사람의 수동 개입이 항상 우선합니다. 여러분의 규칙 두 가지도 같습니다: ① 실행 전 멘토의 리모컨 확인, ② 코드에는 반드시 정지·안전 자세 복귀까지 포함. 35kg가 넘어지는 사고는 장비를 넘어 사람의 사고입니다 — "멈추는 코드가 먼저"는 이번 주의 제1원칙입니다.

2.5 자율학습 미션 — 몸으로 익히는 균형 (월·화)

이 장의 미션은 로봇 없이 진행하는 개념·체감 미션입니다. 답과 소감을 학습일지에 기록합니다 — 수요일 퀴즈 출제 범위입니다.

미션 ① 관절 지도 그리기 (필수)

2.1절의 표를 덮고, 사람 형태의 간단한 스케치에 부위별 관절(다리 6자유도 포함)을 표시해 봅니다. 그린 뒤 표와 대조하고, 자기 몸으로 각 축을 실제로 움직여 봅니다 — 특히 발목의 두 방향(발끝 들기/발바닥 기울이기)을 의식적으로.

미션 ② 지지 영역 체감 실험 (필수 · 사람 실험)

안전한 곳에서 다음을 해 보고, 각각 지지 영역과 무게중심의 언어로 한 줄씩 설명해 봅니다.

- ㉠ 두 발을 어깨너비로 벌리고 서기 vs 두 발을 붙이고 서기 — 어느 쪽이 옆 방향으로 안정적인가?
- ㉡ 한 발로 서기 — 몸(특히 팔과 상체)이 무엇을 하기 시작하는가? 발바닥 안에서는 무엇이 느껴지는가?
- ㉢ 서서 상체만 천천히 앞으로 기울이기 — 어느 순간 발가락에 힘이 들어가고, 더 기울이면 무엇이 나가는가?

㉢에서 발이 저절로 나가는 순간이 바로 "자세 보상의 한계 → 스텝"의 전환입니다. 로봇 제여기가 하는 판단을 여러분의 신경계가 방금 시연한 것입니다.

미션 ③ Go2 ↔ G1 대응표 완성 (필수)

Go2 주간에서는	G1 주간에서는?
topic list로 토픽 지도부터 그렸다	() ※ 힌트: 3장
Gazebo에서 검증 후 실기체	()
StandUp → 대기 → Move → StopMove	() ※ 상태 전이 관점에서
잘못되면 리모컨이 코드보다 우선	() ※ 무엇이 더 엄격해지는가

미션 ④ 상태 전이 판별 연습 (필수)

다음 상황이 안전한 전이인지, 왜 그런지 답해 봅니다 — ㉦ 행어 거치·준비 자세에서 기립 명령 ㉧ 앞드린 상태에서 곧바로 보행 명령 ㉨ 보행 중 비상정지 ㉩ 기립 상태에서 팔 모션 명령. (힌트: "현재 상태에서 그 명령이 의미가 있는가"를 기준으로)

도전 미션 ★ 공식 사양 조사 (자율학습)

본 교재는 수치를 대략적으로만 다뤘습니다. Unitree 공식 문서에서 G1의 전체 자유도(우리 구성 기준), 키·무게, 팔의 자유도 수를 찾아 출처와 함께 학습일지에 정리합니다. Go2 주간에 이 미션을 해 본 조원이 있다면 "어디서 찾는지"를 조에 공유 — 문서 찾는 능력도 인수인계 대상입니다.

이번 장의 산출물 정리 2장을 마치면 학습일지에 남아 있어야 할 것 — ① 관절 지도 스케치(표와 대조 완료) ② 균형 체감 실험 ㉠~㉢의 한 줄 설명 ③ Go2 대응표 ④ 상태 전이 판별 답 ⑤ 조별 안전 물리 설명 한 줄. 수요일 퀴즈는 이 다섯 가지에서 출발합니다.

2.6 자가 점검

다음 질문에 막힘없이 답할 수 있으면 3장(DDS 통신과 인터페이스)으로 넘어갑니다.

- ① 다리 6 자유도의 구성(고관절 3 + 무릎 1 + 발목 2)과 각 축의 역할을 말할 수 있는가?
- ② 사족과 달리 휴머노이드에 발목 관절이 필수인 이유는? (점 vs 면)
- ③ support polygon 과 무게중심의 관계로 "넘어짐"을 설명할 수 있는가?
- ④ 자세 보상과 스텝은 각각 언제 쓰이는 균형 전략인가?
- ⑤ 행어 거치가 기본인 이유를 균형 제어의 원리와 연결해 설명할 수 있는가?
- ⑥ 상태 기계 관점에서 "엎드린 로봇에 보행 명령"이 왜 거부되어야 하는가?
- ⑦ 손끝의 위치와 방향을 자유롭게 정하려면 최소 몇 자유도가 필요하고, 그것이 팔 구성과 어떻게 이어지는가?
- ⑧ 보행 반경 2 m 라는 숫자를 무게중심 높이·질량의 언어로 정당화할 수 있는가?

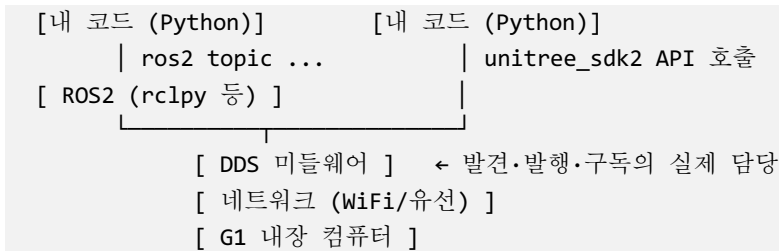
다음 장 예고 구조를 알았으니 이제 데이터를 봅니다. 3장에서는 ROS2의 밑단인 DDS를 만나고, G1의 상태·명령 토픽을 시뮬레이터에서 조사합니다 — Go2 주간의 탐사 루틴이 세 번째로, 그리고 가장 깊게 재등판합니다.

3. DDS 통신 구조와 G1의 인터페이스

2장에서 "무엇이 있는지"를 배웠다면, 이 장에서는 그 부품들과 대화하는 통로를 다룹니다. 목표는 세 가지입니다. 첫째, Week 1부터 써 온 ROS2 통신의 밑단인 DDS가 무엇인지 알고, G1에서 만나는 "두 가지 창구"(ROS2 토픽 / SDK 직접 통신)를 구분할 수 있게 되는 것. 둘째, G1의 핵심 데이터 흐름을 방향과 함께 지도로 그리는 것. 셋째, 미션 A로 시뮬레이터의 상태 토픽을 직접 해부해 — 수십 개 관절의 데이터가 실제로 어떻게 생겼는지 눈으로 확인하는 것입니다.

3.1 DDS — ROS2의 밑단을 만나다

Week 1에서 "ROS2는 같은 네트워크의 노드를 자동 발견한다"고 배웠습니다. 그 마법의 정체는 DDS(Data Distribution Service)입니다. ROS2의 토픽 발행·구독은 실제로는 DDS라는 산업 표준 통신 미들웨어 위에서 동작하며, Unitree SDK는 이 DDS 계층을 직접 사용해 로봇과 통신합니다. 층으로 그리면 다음과 같습니다.



그래서 G1에서는 두 가지 얼굴을 만나게 됩니다.

경로	설명
ROS2 토픽으로 보기	unitree_ros2 드라이버를 통해 상태·명령이 ROS2 토픽으로 노출 — Week 1의 CLI 조사법 그대로 사용
SDK로 직접 통신	unitree_sdk2가 DDS로 로봇과 직접 대화 — high-level API 예제가 이 방식

둘은 대립하는 것이 아니라 같은 DDS 위의 두 창구입니다. 이번 주 예제는 SDK 경로를 주로 쓰되, 상태 관찰은 ROS2 토픽으로도 병행합니다. 비유하자면 — 같은 은행(DDS)에 창구 방문(ROS2 토픽·CLI로 관찰하기 좋음)과 앱(SDK·코드에서 명령하기 좋음)이 함께 있는 셈이고, 어느 쪽으로 입금해도 같은 계좌에 반영됩니다.

왜 지금 DDS를 알아야 하는가

- **도구 선택의 눈** : "관찰은 ROS2 CLI로, 동작 명령은 SDK 예제로"라는 이번 주의 작업 분담이 자연스러워집니다
- **트러블슈팅의 눈** : ROS_DOMAIN_ID(Week 1)와 SDK의 네트워크 인터페이스 지정이 왜 각각 필요한지 — 둘 다 DDS의 발견 범위를 다루는 장치임을 알게 됩니다

- **연구로의 다리** : low-level 제어 연구(WBC-RL)로 가면 SDK/DDS 경로가 표준 도구가 됩니다 — 이번 주는 그 입구의 문패를 읽는 단계

이번 주의 실제 사용 장면으로 보면

하고 싶은 일	쓰는 창구	이유
상태 토픽 주기·내용 조사 (미션 A)	ROS2 토픽 + CLI	Week 1 조사법을 그대로 재사용
행어 위 기립·팔 모션 실행 (5·6장)	SDK 예제	high-level API가 이 경로로 제공됨
모션 중 관절 반응 관찰 (7장)	둘 다 병행	명령은 SDK, 관찰·녹화는 ROS2 — 두 창구를 동시에 여는 것이 표준 작업대

3.2 핵심 데이터 흐름 — 방향부터 읽는다

Go2 주간과 같은 독법입니다 — 이름보다 방향 먼저. "내 코드 → G1"은 명령, "G1 → 내 코드"는 상태입니다.

데이터	방향	설명
low state (관절·IMU 상태)	G1 → 내 코드	수십 개 관절의 각도·속도·토크 + IMU — "로봇의 지금"을 읽는 창
모션/보행 명령	내 코드 → G1	high-level API 호출이 내부적으로 명령 메시지로 변환되어 전달
모드/상태 보고	G1 → 내 코드	현재 제어 모드·에러 상태 — 실기체 조작 전 반드시 확인

읽기와 쓰기의 비대칭 low state는 이번 주 내내 읽기 전용입니다. 관절 상태를 "읽는" 것은 언제나 안전하지만, 관절에 직접 "쓰는"(low-level 명령) 것은 연구 단계의 영역입니다(2.4절의 금지선). 같은 채널 구조라도 방향에 따라 위험이 완전히 다르다는 감각 — 휴머노이드 주간의 핵심 안전 감각입니다.

low state 미리 해부 — 미션 A에서 만날 것

low state 메시지는 "관절 데이터의 배열"이 중심입니다. 배열의 각 칸이 관절 하나이고, 칸마다 대략 다음 값들이 들어 있습니다(정확한 필드명은 미션 A에서 interface show로 확인).

```
motor_state[i]      # i번째 관절 (배열 길이 = 관절 수 → 미션 A 질문 ①)
├─ q      : 현재 각도          └─ dq : 각속도
├─ tau    : 토크(힘의 회전 성분) └─ ... : 온도 등
imu_state           # 몸통 기울기·각속도 - 2장의 균형 이야기가 숫자로
```

여기서 2장이 그대로 이어집니다 — 발목 관절의 q가 미세하게 계속 변한다면 그것이 "자세 보상"의 숫자 버전이고, imu의 기울기 값이 그 원인 신호입니다. 미션 A에서 이 대응을 직접 확인합니다.

상태 보고는 왜 이렇게 빠른가

휴머노이드의 상태 토픽은 갱신 주기가 매우 빠른 편입니다(정확한 값은 미션 A에서 측정). 이유는 2장에서 배운 균형의 성질에 있습니다 — 높은 무게중심 때문에 기울어짐이 빠르게 커지므로, 제어기는 그만큼 자주 상태를 읽고 자주 보정해야 합니다. "상태 갱신 주기"는 곧 "그 로봇의 균형이 얼마나 절박한가"의 지표이기도 합니다. turtlesim → Go2 → G1로 오며 주기가 어떻게 변하는지 비교하는 것이 미션 A 질문 ②입니다.

모드/상태 보고 — 조작 전 확인 루틴

세 번째 흐름(모드/상태 보고)은 이번 주 안전 절차의 데이터 버전입니다. 실기체는 시뮬레이터든, 동작 명령을 보내기 전 확인하는 순서를 지금 루틴으로 만들어 둡니다.

확인 항목	무엇을 보는가	이상하면
현재 모드	지금 어떤 상태인가 (거치/준비/기립 등) — 2.4절 상태 기계의 현재 위치	그 상태에서 유효한 명령만 — 전이 순서 준수
에러 상태	에러 코드·경고가 떠 있는가	명령 중단, 멘토 보고 — 에러 위에 명령을 쌓지 않는다
배터리 온도	여유가 있는가 (실기체)	낮으면 정리 모드 — Go2 주간과 동 일 원칙

이 루틴이 5장의 코드에서 "상태 확인 → 명령 → 전이 대기"라는 형태로 다시 나타납니다. 사람의 체크리스트와 코드의 체크가 같은 구조라는 것 — 좋은 로봇 시스템의 특징입니다.

3.3 조사 도구 복습 + G1 특화 요령

탐사 5단계 루틴과 Go2 주간의 추가 도구(hz·bw---no-arr·bag)는 그대로 통합합니다. G1에서 특히 유용한 요령 세 가지만 더합니다.

요령	왜 G1에서 특히 유용한가
echo --once + 출력 저장	관절 배열이 길어 흐르는 화면으로는 못 읽습니다 — 한 번만 받아 파일로 저장해 천천히 쉵니다 (> state.txt)
grep -c 로 개수 세기	저장한 출력에서 반복되는 필드를 세면 관절 수가 나옵니다 — 교재 ① 3장의 grep이 실전 투입
bag 30초 녹화	행어 위 균형 동작을 녹화해 두면 관절 미세 움직임을 나중에 재생· 분석 — "자세 보상"의 증거 데이터

QoS 재확인 Go2 주간에 배운 복병이 여기에도 있습니다 — list에 있고 hz도 뛰는데 echo만 조용하면 QoS 불일치 의심. "안 나온다 = 없다"가 아닙니다. hz로 생존 확인부터.

SDK 예제 실행의 한 가지 관문 — 네트워크 인터페이스

SDK 경로의 예제들은 실행 시 "어느 네트워크 인터페이스로 DDS 통신을 할지"를 지정하는 경우가 많습니다(실기체는 로봇이 연결된 유선/무선 인터페이스, 시뮬레이터는 내 컴퓨터 내부 통신). 정확한 실행 인자는 저장소 README가 기준이며, 여기서 기억할 것은 하나 — "SDK 예제가 아무 반응이 없다"의 단골 원인이 이 인터페이스 지정 착오라는 사실입니다. ROS2의 ROS_DOMAIN_ID와 함께, 둘 다 "DDS가 어디까지 보고 듣는가"를 정하는 손잡이라고 이해하면 지도가 완성됩니다.

3.4 자율학습 미션 A — 상태 토픽 조사 (월·화, 시뮬레이터)

배부된 G1 교육용 저장소를 클론하고 시뮬레이터를 띄운 뒤, Week 1의 조사법을 적용합니다. 답을 학습일지에 기록하세요 — 수요일 퀴즈 범위입니다.

미션 ① 환경 준비와 첫 탐사 (필수)

```
cd ~/robot_ws/src && git clone <멘토 배부 G1 교육용 저장소 주소>
cd ~/robot_ws && colcon build && source install/setup.bash

# 시뮬레이터 실행 후, 다른 터미널에서
ros2 topic list
ros2 topic echo <low state 토픽> --once > state.txt # 파일로 저장
less state.txt # 천천히 읽기 (q로 종료)
ros2 topic hz <low state 토픽> # 상태 보고 주기는 몇 Hz?
ros2 interface show <low state 타입> # 공식 양식 확인
```

조사 질문 — 학습일지에 기록:

- ① low state 에 담긴 관절 데이터는 몇 개이며, 각 관절에 어떤 값(각도·속도·토크)이 있는가? — 2.1 절의 부위별 자유도 합과 맞아떨어지는지 검산
- ② 상태 보고 주기는 Go2-turtlesim 과 비교해 어떤가? 왜 휴머노이드는 더 빠른 상태 갱신이 필요할까? (힌트: 3.2 절)
- ③ DDS 의 자동 발견 때문에 생기는 실습실 문제와 그 해결책은? (힌트: Week 1 FAQ 의 ROS_DOMAIN_ID)

미션 ② 관절 하나를 골라 추적하기 (필수)

state.txt에서 관절 하나(예: 왼쪽 발목)를 골라, 시뮬레이터의 로봇이 서 있는 동안 echo를 다시 열어 그 관절의 q(각도)가 시간에 따라 어떻게 변하는지 10초간 관찰합니다. 완전히 고정입니까, 미세하게 흔들리니까? — 관찰 결과를 2.2절의 "자세 보상" 개념과 연결해 두 문장으로 설명합니다. (어느 칸이 어느 관절인지 저장소 문서/멘토 안내로 확인 — "배열 순서를 문서에서 찾는 것"도 조사 능력입니다)

미션 ③ 나만의 데이터 흐름 지도 (필수)

데이터	실제 토픽/채널 이름	타입	주기(Hz)
low state			
모션/보행 명령			
모드/상태 보고			

3.2절의 표를 실측으로 완성합니다. SDK 경로로만 존재해 ROS2 토픽에 안 보이는 항목이 있다면 "SDK 경로"라고 적습니다 — 두 창구의 구분(3.1절)이 실측으로 확인되는 순간입니다.

기록 습관 미션 ①~③의 명령과 출력은 복사·붙여넣기가 아니라 "내가 무엇을 확인했는지" 한 줄 해석과 함께 남깁니다 — 예: "hz 결과 ○○○Hz, Go2의 △△Hz보다 빠름 → 3.2절의 균형 절박함과 일치". 해석이 붙은 기록만이 수요일 Q&A에서 토론 재료가 됩니다.

미션 ④ 30초 녹화 — 균형의 증거 (필수)

```
cd ~/robot_study/g1_week
ros2 bag record -o balance_state <low state 토픽> # 30초 후 Ctrl+C
ros2 bag info balance_state # 메시지 수 ≈ 주기 × 30초 계산
```

미션 ⑤ 관절 수 계산 대조 (필수 · 조 단위)

조원 각자가 센 관절 수(질문 ①)와 2.1절 부위별 자유도 합, 그리고 도전 미션(2장)에서 조사한 공식 사양의 자유도를 한자리에 놓고 대조합니다. 세 숫자가 다르다면 — 손 관절 포함 여부, 시뮬레이터 모델 구성 차이 등 "차이의 이유"를 찾아내는 것이 이 미션의 진짜 목표입니다. 결론을 조 채널에 세 줄로 정리합니다.

도전 미션 ★ 관절 흔들림 크기 재보기 (자율학습)

미션 ②의 관찰을 숫자로 만듭니다 — 녹화한 bag을 재생하며(또는 echo 출력을 파일로 모아) 선택한 관절 q의 최댓값과 최솟값을 찾아 "흔들림 폭"을 계산해 봅니다. 방법은 자유(눈으로 읽기 / grep과 sort / 간단한 파이썬). 이 폭이 4장에서 보행 파라미터를 바꿀 때 어떻게 변하는지 비교하면, 7장 미션의 관찰 항목 하나를 미리 확보한 셈입니다.

이번 장의 산출물 정리 3장을 마치면 학습일지·repo에 남아 있어야 할 것 — ① state.txt(저장한 low state 출력)와 관절 수 계산 메모 ② 조사 질문 ①~③의 답 ③ 데이터 흐름 지도 표(실측 완성본) ④ balance_state bag과 계산 결과 ⑤ 조 채널의 관절 수 대조 세 줄. 수요일 퀴즈와 4장의 파라미터 관찰이 전부 이 위에서 시작됩니다.

3.5 자가 점검

다음 질문에 막힘없이 답할 수 있으면 4장(MuJoCo 시뮬레이션)으로 넘어갑니다.

- ① DDS 는 무엇이고, ROS2·Unitree SDK 와 어떤 층 관계인가?
- ② "두 창구"(ROS2 토픽 / SDK)는 각각 무엇에 좋고, 이번 주 어떻게 분담하는가?
- ③ 세 가지 핵심 데이터 흐름을 방향과 함께 말할 수 있는가?
- ④ low state 가 읽기 전용인 이유를 안전의 언어로 설명할 수 있는가?
- ⑤ 휴머노이드의 상태 갱신이 빠른 이유를 2 장의 균형 원리와 연결할 수 있는가?
- ⑥ 관절 배열에서 특정 관절을 찾으려면 무엇을 참조해야 하는가?

다음 장 예고 데이터의 지도를 그렸으니 이제 무대를 준비합니다. 4장에서는 수요일 오후의 주 무대인 MuJoCo 시뮬레이션을 다룹니다 — 왜 휴머노이드는 Gazebo가 아니라 MuJoCo인지, 그리고 보행 파라미터 관찰이라는 이번 주 고유의 실험이 시작됩니다.

4. MuJoCo 시뮬레이션 — 수요일 오후의 주 무대

이번 장은 수요일 오후 실습의 무대인 MuJoCo 시뮬레이션을 다룹니다. 목표는 세 가지입니다. 첫째, 왜 휴머노이드 주간에는 Gazebo가 아니라 MuJoCo를 쓰는지 — 접촉 계산이라는 이유를 설명할 수 있게 되는 것. 둘째, 화요일까지 환경 준비를 마쳐 수요일에 실습만 남겨 두는 것. 셋째, 이번 주 고유의 실험인 보행 파라미터 관찰을 체크포인트를 따라 완주하고, "파라미터 X를 바꾸면 Z가 일어난다"라는 표를 손에 쥐는 것입니다 — 이 표가 금요일 미션의 근거 자료가 됩니다.

4.1 왜 MuJoCo인가

MuJoCo는 접촉(contact)이 많은 동역학을 빠르고 정확하게 계산하는 물리 시뮬레이터로, 휴머노이드·강화학습 연구의 사실상 표준입니다. 발이 땅을 딛고 미는 힘의 계산이 보행 시뮬레이션의 핵심인데, MuJoCo가 특히 이 부분에 강합니다. 연구실의 sim-to-real 연구도 이 위에서 시작됩니다. G1 주간의 시뮬레이션 원칙도 동일합니다 — 시뮬레이션에서 검증되지 않은 것은 실기체에서 시도하지 않습니다. 휴머노이드에서는 이 원칙이 더욱 절대적입니다.

"접촉이 어렵다"는 말의 뜻

물리 시뮬레이션에서 허공을 나는 물체의 운동은 쉬운 문제입니다. 어려운 것은 닿는 순간입니다 — 발바닥이 지면에 닿고, 눌리고, 미끄러질 듯 버티는 짧은 순간에 힘이 급격히 변하며, 이 계산이 조금만 어긋나도 시뮬레이션 속 로봇은 지면을 뚫고 들어가거나 튕겨 오릅니다. 2장의 언어로 말하면 — 휴머노이드의 균형은 "발바닥이라는 면과 지면의 접촉" 위에서 성립하므로, 접촉을 잘 푸는 시뮬레이터가 곧 휴머노이드를 잘 푸는 시뮬레이터입니다. Go2 주간의 Gazebo가 틀린 선택이었던 것이 아니라, 문제의 성격에 따라 도구를 고른 것입니다 — "과제에 맞는 시뮬레이터 선택"도 연구 기술의 하나입니다.

Gazebo와 무엇이 다른가 — 한눈 비교

	Gazebo (Go2 주간)	MuJoCo (이번 주)
강점	센서·월드·ROS2 연동이 풍부한 종합 환경	접촉 동역학의 속도·정확도 — RL 학습 무대
화면	월드 속 로봇 + 환경(장애물·건물)	로봇과 지면 중심의 간결한 뷰어
이번 주 용도	(지난 주) 주행·LiDAR 환경 시뮬레이션	보행 그 자체의 관찰·파라미터 실험

rviz2는 그대로 시뮬레이터가 바뀌어도 관찰 도구는 그대로입니다 — 시뮬레이터가 ROS2 토픽을 발행하는 구성이라면(저장소 기준) rviz2-CLI 조사법을 지난주처럼 쓸 수 있습니다. "시뮬레이터는 무대, ROS2는 방송"이라는 구분이 여기서도 유효합니다.

4.2 환경 준비 (화요일까지 완료)

```
# MuJoCo 및 예제 의존성 설치 (배부 저장소 README의 목록대로)
pip install mujoco <기타 의존성>

# G1 모델로 보행 예제 실행
cd ~/robot_ws/src/<g1 교육용 저장소>
python <보행 예제 스크립트> # MuJoCo 뷰어에 G1이 등장해 걷는다
```

실행에 성공했다면 화요일의 목표는 달성입니다 — 파라미터 실험은 수요일에 합니다. 확인 기준 세 가지:

- ① 뷰어 창에 G1 모델이 나타난다, ② 로봇이 넘어지지 않고 걷는다, ③ 터미널에 빨간 오류가 반복 출력되지 않는다.

막히면 MuJoCo 뷰어가 열리지 않는 문제는 대부분 GPU/그래픽 드라이버 또는 WSL2 환경 이슈입니다. 오류 메시지를 캡처해 조 채널에 공유하고, 해결이 안 되면 수요일 오전 Q&A에 가져옵니다. WSL2 사용자는 Week 1 FAQ의 GUI 항목을 먼저 확인하세요.

뷰어 조작 — 이것만 알면 관찰이 시작된다

조작	방법 (구성에 따라 다를 수 있음 — 뷰어 도움말 확인)
시점 회전 / 이동 / 줌	마우스 드래그 / 우클릭(또는 Shift) 드래그 / 휠
일시정지 / 재개	Space — 발이 닿는 순간에 멈춰 놓고 관찰할 때
슬로모션	재생 속도 조절 — 스텝 순간 분석의 핵심 도구
리셋	예제 재실행(또는 뷰어의 리셋) — 넘어지면 당황하지 말고 처음부터

지난주 Gazebo의 RTF-sim time 개념이 여기서도 그대로 적용됩니다 — 시뮬레이션 세계의 시간은 별도의 시계로 흐르고, 성능이 부족하면 느리게 계산됩니다. 슬로모션은 "고장"이 아니라 관찰 도구라는 점만 기억하면 됩니다.

4.3 시뮬레이션 실습 체크포인트 (수요일 오후)

No	체크포인트	성공 기준
①	보행 예제 실행 — G1이 넘어지지 않고 걷는 기본 상태 확인	1분 이상 안정 보행, 화면 캡처
②	뷰어 조작(회전·확대·슬로모션)으로 발 닿는 순간의 지지 영역 변화 관찰	"한 발 지지 ↔ 두 발 지지"의 교대를 슬로모션으로 지목할 수 있다
③	보행 파라미터(속도·보폭·주기 등, 예제가 노출하는 값)를 하나씩 바꿔 실행 — 무엇이 언제 넘어지게 만드는가?	파라미터 3개 이상, 각 2회 이상 변경 실험
④	관찰 기록: "파라미터 X를 값 Y로 바꾸면 Z가 일어난다"를 표로 정리 → 7장 미션의 기초 데이터	4.4 양식 표가 5행 이상 채워짐

No	체크포인트	성공 기준
⑤	여유가 되면: 상체 모션 예제(팔 동작)를 실행해 보행과 동시에 걸었을 때의 균형 변화 관찰	"팔 동작이 균형에 주는 영향" 한 줄 관찰

권장 시간 배분(14~18시): ① 14:00~14:30(환경 재확인·조원 지원) → ② ~15:20 → ③·④ ~17:20(이번 주의 본론 — 가장 길게) → ⑤·정리 ~18:00. 지난주와 같은 원칙 — 속도보다 순서, 그리고 "조 전원이 되는 것"이 성공 기준입니다.

수요일 빈출 상황 — 우선 확인할 것

증상	우선 확인
뷰어가 안 뜨거나 검게 나옴	GPU 드라이버(nvidia-smi) · WSL2 GUI(Week 1 FAQ) · 원격 접속 여부
로봇이 시작하자마자 넘어짐	예제를 처음부터 재실행(초기 자세 문제) · 파라미터를 기본값으로 되돌렸는가
파라미터를 바꿨는데 변화가 없음	저장 후 재실행했는가 · 다른 파일/변수를 바꾸고 있지 않은가 — 기본값 대비 원칙
시뮬레이션이 눈에 띄게 느림	슬로모션 설정 확인 · 성능 부족(다른 프로그램 종료) — 느린 것 자체는 결과를 왜곡하지 않음(시물 시간 별도)
python 모듈을 찾을 수 없음	pip 설치를 한 환경과 실행 환경이 같은가 — 교재 ④ 5장의 가상환경 확인

4.4 파라미터 관찰표 양식 (③·④에서 사용)

파라미터	기본값 → 변경값	관찰된 현상	넘어짐 여부
(예) 보행 속도	0.3 → 0.8	상체 흔들림 증가, 스텝 빨라짐	X

한 번에 하나의 파라미터만 바꾸고, 반드시 기본값 대비로 기록합니다. 이 표가 금요일 과제 1의 근거 자료가 됩니다.

좋은 기록과 나쁜 기록

- 나쁜 기록 : "속도를 올리니 이상해짐" — 무엇이 얼마나, 언제부터인지 없음
- 좋은 기록 : "속도 0.3→0.8: 상체 좌우 흔들림이 눈에 띄게 커지고, 방향 전환 시 두 번 비틀거림. 넘어짐 X" — 다음 사람이 재현할 수 있는 기록
- 가장 좋은 기록 : 여기에 "왜"의 가설 한 줄 — "스텝 주기가 그대로인데 이동량만 커져 발이 따라가지 못하는 듯" — 맞든 틀리든 가설이 있어야 다음 실험이 생깁니다

경계를 찾는 요령 — 이분 탐색

"무엇이 언제 넘어지게 만드는가"(체크포인트 ③)의 효율적인 방법은 경계 찾기입니다. 기본값에서 크게 바꿔 넘어지는 값을 하나 찾은 뒤, 기본값과 그 사이의 중간값을 시도하며 "안 넘어지는 최대값"을 좁혀 갑니다 — 교재 ①의 이분 탐색이 실험 설계로 재등장하는 순간입니다. 각 파라미터의 "안전 경계"를 숫자로 확보해 두면, 7장에서 튜닝 목표를 정할 때 지도가 됩니다.

파라미터는 왜 균형을 흔드는가 — 해석의 실마리

관찰표를 채우다 보면 패턴이 보이기 시작합니다. 해석의 실마리 두 가지만 미리 쥐여 드립니다 — 첫째, 속도·보폭을 키우는 것은 매 스텝에서 무게중심이 더 멀리, 더 빨리 이동한다는 뜻이므로 그만큼 발이 제 때 따라 들어와야 하고(스텝 타이밍의 여유 감소), 둘째, 발 높이·주기를 바꾸는 것은 "한 발 지지 시간"의 길이를 바꾸므로 가장 아슬아슬한 구간의 길이가 달라집니다. 여러분의 관찰이 이 실마리와 맞는지 다른지 — 다른 경우가 더 흥미로운 발견입니다. 정답 맞히기가 아니라 현상과 가설을 잇는 연습이 목적입니다.

4.5 관찰의 눈 — 무엇을 볼 것인가

- 발이 언제 어디를 딛는가 (스텝 타이밍과 위치가 균형의 전부입니다)
- 상체가 어떻게 흔들리는가 (허리·팔의 미세한 보상 동작 찾기)
- 넘어질 때는 "왜" 넘어졌는가 — 무게중심이 지지 영역을 벗어난 순간을 슬로모션으로 포착

세 항목이 전부 2장의 개념 확인입니다 — 지지 영역(2.2), 자세 보상(2.2), 무게중심 이탈(2.2). 교재를 읽을 때는 개념이었던 것이 화면에서는 매 걸음 일어나는 사건이 됩니다. 관찰 ③이 특히 중요합니다: 넘어짐의 순간을 슬로모션으로 되돌려 "무게중심이 어느 방향으로 지지 영역을 벗어났는지"를 지목할 수 있으면, 이번 주의 이론은 전부 여러분의 것입니다.

외란 실험은 여기서 2장에서 예고한 실험 — 로봇을 살짝 밀면 어떻게 되는가 — 를 시뮬레이터에서 안전하게 해 볼 수 있습니다(예제가 외란 기능을 제공하는 경우, 방법은 README/멘토 확인). 작은 외란은 자세 보상으로, 큰 외란은 스텝으로 회복하는지, 더 크면 넘어지는지 — 2.2절 "두 가지 무기"의 실증입니다.

4.6 자율학습 미션 — 시뮬레이션 준비 (월·화)

미션 ① README 통독 후 설치 (필수)

저장소 README를 처음부터 끝까지 통독한 뒤 설치를 시작합니다(읽으며 따라 하지 말 것 — 전체 그림 먼저). 설치 후 4.2절의 확인 기준 세 가지를 점검하고 결과를 학습일지에 기록합니다.

미션 ② 첫 실행 캡처 (필수)

걷는 G1의 뷰어 화면을 캡처해 학습일지에 붙이고 조 채널에 공유합니다 — 화요일 밤까지 조 환경 현황판(설치 완료/막힌 점 한 줄)을 완성하는 것은 지난주와 동일한 조 단위 과제입니다.

미션 ③ 지지 영역 예습 관찰 (필수)

설치 확인을 겸해 보행을 1분간 관찰하며, 2장 미션 ②(사람 실험)에서 몸으로 느꼈던 것 — 한 발 지지의 아슬아슬함, 상체의 보상 — 을 화면에서 찾아봅니다. "내 몸에서는 ○○였는데 로봇 화면에서는 △△로 보인다" 한 문장을 학습일지에.

미션 ④ 파라미터 목록 만들기 (필수)

예제 코드/설정 파일을 열어 바꿀 수 있는 보행 파라미터의 목록(이름·기본값·추정 의미)을 표로 만들어 둡니다 — 수요일 체크포인트 ③에서 바로 실험에 들어가기 위한 준비입니다. 의미를 모르는 파라미터는 "?"로 남겨 수요일 Q&A 질문거리로.

미션 ⑤ 파라미터 분담 계획 (필수 · 조 단위)

미션 ④에서 만든 파라미터 목록을 조에서 취합해, 수요일 체크포인트 ③의 실험을 분담합니다 — 조원별로 담당 파라미터를 나누고, 전원이 같은 양식(4.4)으로 기록해 오후가 끝날 때 하나의 표로 합칩니다. 10명이 각자 같은 실험을 반복하는 것보다, 나눠 맡아 합치는 편이 관찰표의 폭이 훨씬 넓어집니다 — 분담표를 화요일 밤 조 채널에 공유합니다.

도전 미션 ★ 3장과 연결하기 (자율학습)

시뮬레이터가 떠 있는 동안 3장의 조사법을 적용해 봅니다 — low state 토픽이 시뮬레이션에서도 흐르는지, 흐른다면 보행 중 발목 관절 q 의 변화 폭이 서 있을 때(3장 도전 미션)와 어떻게 다른지. "시뮬레이터가 바뀌어도 조사법은 같다"를 스스로 증명하는 미션입니다.

이번 장의 산출물 정리 4장을 마치면 남아 있어야 할 것 — ① 걷는 G1 첫 실행 캡처 ② 파라미터 목록 표(미션 ④) ③ 채워진 파라미터 관찰표(체크포인트 ④, 5행 이상) ④ 넘어짐 순간 슬로모션 관찰 한 줄 ⑤ 조 환경 현황판. 이 중 ③이 금요일 과제 1의 근거 자료이자 7장 튜닝 목표의 출발점입니다.

목요일 예고 수요일의 관찰표는 목요일 미션 설계일의 재료입니다 — 7장 미션의 과제 1(보행 파라미터 튜닝)은 "관찰표에서 목표 파라미터와 목표값을 골라 오는 것"에서 시작하니, 관찰표를 조 repo에 커밋해 두고 목요일에 함께 펴 놓으세요.

4.7 자가 점검

다음 질문에 막힘없이 답할 수 있으면 5장(high-level API)으로 넘어갑니다.

- ① 휴머노이드 시뮬레이션에서 접촉 계산이 왜 핵심이고, MuJoCo 는 왜 그 표준인가?
- ② Gazebo 와 MuJoCo 의 용도 차이를 "문제의 성격에 따른 도구 선택"으로 설명할 수 있는가?
- ③ 수요일 체크포인트 ①~⑤의 순서와 성공 기준을 말할 수 있는가?
- ④ "좋은 관찰 기록"의 세 요소(재현 가능한 서술·기본값 대비·가설 한 줄)는 무엇인가?
- ⑤ 파라미터의 안전 경계를 효율적으로 찾는 방법(이분 탐색)을 설명할 수 있는가?
- ⑥ 관찰의 눈 세 가지가 각각 2 장의 어느 개념과 대응하는가?

다음 장 예고 관찰만 하던 로봇에게 이제 말을 겁니다. 5장에서는 high-level API로 기립·모션을 코드로 명령합니다 — 3장의 "상태 확인 → 명령 → 전이 대기" 리듬과 2장의 상태 기계가 드디어 코드가 됩니다.

5. high-level API — 코드로 모션 명령하기

4장까지가 관찰이었다면 이번 장부터는 대화입니다. high-level API로 G1에게 기립·보행·상체 모션을 코드로 명령하는 법을 다룹니다. 목표는 세 가지 — 첫째, 명령을 "함수 호출"이 아니라 "상태 전이"로 이해하는 것(2장 상태 기계의 코드 버전). 둘째, "시작은 Damp, 이상 시 즉시 Damp"라는 이번 주 안전 습관을 코드 수준에서 몸에 붙이는 것. 셋째, 미션 B로 상체 모션 예제를 실행 전에 완전히 읽어내는 것 — "실행 전 설명 가능" 원칙의 G1 버전입니다.

5.1 대표 명령 흐름 — 상태 전이로 읽는다

G1의 high-level API는 "상태 전이"의 개념으로 이해하면 쉽습니다. 로봇은 항상 어떤 모드에 있고, 우리는 허용된 순서로만 다음 모드로 넘어갈 수 있습니다 (정확한 함수명·시그니처는 배부 저장소 예제 참조).

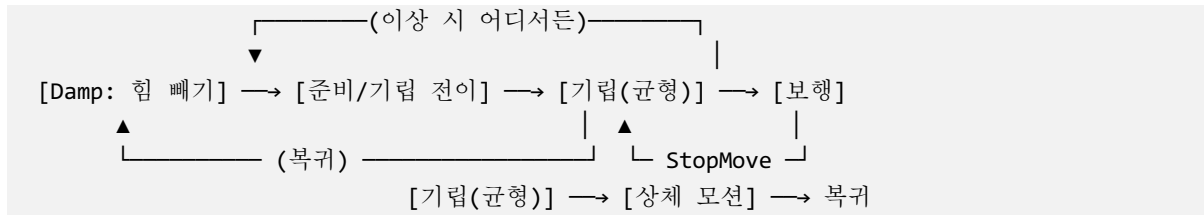
명령 (예)	동작
Damp()	전 관절 힘 빼기 — 모든 절차의 시작이자, 이상 시 복귀 지점
StandUp() 계열	기립 자세로 전이 (행어 거치 상태에서 연습)
Move(vx, vy, vyaw)	보행 속도 명령 — Go2와 같은 3요소, 그러나 훨씬 보수적인 값으로
StopMove()	보행 정지 — 코드에서 가장 자주 호출해야 할 함수
상체 모션/제스처 API	손 흔들기 등 팔 동작 시퀀스 — 7장 미션의 재료

"보수적인 값"이란 얼마인가 — 속도 감 잡기

Move의 3요소는 Go2 주간 5.3절과 같은 좌표계·단위입니다(몸통 기준, m/s와 rad/s — 회전하면 "앞"도 함께 돕니다). 다른 것은 값의 크기입니다. 두 발 균형 위에서는 같은 속도도 균형 여유를 훨씬 크게 깎아 먹으므로, 이번 주의 시작값은 Go2보다 한 단계 낮게 잡습니다 — 정확한 상한은 멘토가 지정하며, 아래는 감을 잡기 위한 눈금입니다.

값 (예)	의미	메모
$v_x = 0.1 \sim 0.2$	천천히 걷기 수준의 전진	이번 주 시뮬레이션 실험의 기본 영역
$vyaw = 0.2 \sim 0.3$	완만한 제자리 회전	회전은 균형 여유를 가장 빨리 깎는 명령 — 가장 보수적으로
실기체	멘토 지정 상한 이내	4장 관찰표의 "안전 경계"보다 훨씬 안쪽에서 시작

모드 다이어그램 — 명령이 다니는 길



화살표가 없는 곳으로는 갈 수 없습니다 — 예를 들어 Damp에서 곧바로 보행으로 가는 화살표는 없습니다. 좋은 API는 잘못된 전이를 거부하지만, "거부당하는 코드"를 쓰지 않는 것이 우리의 목표입니다. 스크립트를 짤 때는 이 그림 위에 내 명령들의 경로를 먼저 그려 보고, 화살표를 따라가는지 확인합니다 — 미션 B의 첫 과제가 정확히 이것입니다.

상체 모션 API — 시퀀스라는 개념

표의 마지막 줄, 상체 모션 API는 이번 주의 주인공입니다. 내부적으로는 "팔·허리 관절들이 시간에 따라 거쳐 갈 목표 자세들의 목록" — 즉 시퀀스를 재생하는 것으로, 예제에는 손 흔들기 같은 완성 시퀀스가 제공됩니다. 여기서 두 가지를 관찰 목표로 잡습니다 — 첫째, 시퀀스 재생 중에도 균형 제어는 계속 돕니다(팔이 움직이면 무게중심이 이동하므로 하체·허리가 보상 — 2.2절의 실시간 버전). 둘째, 시퀀스는 "기립(균형)" 상태에서만 의미가 있습니다 — 모드 다이어그램에서 상체 모션이 기립 옆에 매달려 있는 이유입니다. 7장 과제 2는 이런 시퀀스를 조합·설계하는 일입니다.

안전 습관 모든 스크립트는 "시작은 Damp, 이상 시 즉시 Damp"를 원칙으로 작성합니다. 예제 코드에서 예외 처리(try/except)가 Damp 호출로 끝나는 이유를 찾아보세요. Go2의 "멈추는 코드가 먼저"가 휴머노이드에서는 "힘 빼는 코드가 먼저"로 한 단계 더 근본적이 됩니다 — 정지는 균형 유지를 계속하지만, Damp는 잘못된 힘 자체를 끊습니다(행어 위이므로 안전).

코드 빼대 — "확인 → 명령 → 대기"의 리듬

의사코드 — 실제 함수명은 저장소 예제 기준

- ① 초기화 : 클라이언트 준비 (3장의 SDK 창구, 네트워크 인터페이스 지정)
- ② 상태 확인 : 현재 모드·에러 확인 — 3.2절의 "조작 전 확인 루틴"의 코드 버전
- ③ Damp() : 알려진 안전 상태에서 출발
- ④ 기립 전이 : 명령 → 전이 완료를 기다림(sleep 또는 상태 재확인)
- ⑤ 모션 실행 : 상체 모션/보행 등 이번 목적의 동작
- ⑥ 정리 : StopMove/자세 복귀 → (필요 시) Damp — 끝도 알려진 상태로

※ try/except : 어느 단계에서 실패해도 except에서 Damp로 수렴

Go2 스크립트와 빼대는 같지만 두 가지가 추가되었습니다 — ②의 사전 상태 확인(휴머노이드는 "지금 어떤 모드인가"가 훨씬 중요)과, ※의 예외 처리(중단·실패 시에도 로봇을 알려진 안전 상태로). "정상 경로"만이 아니라 "실패 경로"까지 설계된 코드가 휴머노이드의 표준입니다.

흔한 실수 — G1 버전

실수	무슨 일이 생기고, 어떻게 피하나
상태 확인 없이 명령부터	현재 모드와 안 맞는 명령 — 거부되거나 예측 불가. 항상 ②(상태 확인)부터
전이 완료 전에 다음 명령	전이가 겹쳐 이상 거동 — 대기(sleep/상태 확인)는 생략 불가
예외 처리 없는 스크립트	Ctrl+C 오류 시 로봇이 미지의 상태에 방치 — except는 Damp로 수렴
Go2 감각의 속도 값	휴머노이드의 균형 여유를 초과 — 시작은 한 단계 낮게, 상한은 멘토 지정
행어 없이 실행 상상	이번 주 모든 학생 실행은 행어 거치 전제 — 6장 SOP의 대전제

5.2 자율학습 미션 B — "기립 → 손 흔들기 → 복귀" 예제 읽기 (월·화)

배부 저장소의 상체 모션 예제를 실행하기 전에 코드로 먼저 읽습니다. "실행 전 설명 가능" 원칙의 G1 버전입니다.

미션 ① 예제 해부 (필수 — 원본 질문)

- 이 스크립트의 상태 전이 순서를 그림(모드 다이어그램)으로 그려보라 — Damp에서 시작해 어디를 거쳐 어디로 돌아오는가 (5.1의 그림 위에 경로를 색칠해도 좋습니다)
- 각 전이 사이에 대기(sleep)가 들어가는 이유는 무엇인가?
- 중간에 Ctrl+C로 중단되면 로봇은 어떤 상태에 남는가? 안전한가? — 답을 학습일지에 기록

미션 ② 실패 경로 추적 (필수)

예제에서 예외 처리(try/except)와 관련된 줄을 전부 찾아 표시하고, "어느 단계에서 실패하면 → 어떤 코드가 실행되어 → 로봇이 어떤 상태로 남는가"를 세 가지 시나리오(㉠ 기립 전이 중 실패 ㉡ 모션 실행 중 Ctrl+C ㉢ 통신 끊김)에 대해 각각 한 줄로 적습니다. 예외 처리가 없다면 — 그것을 발견한 것 자체가 중요한 관찰이며, 수요일 Q&A의 1급 질문입니다.

미션 ③ 시뮬레이션에서 실행·검증 (필수 · 수요일)

코드 읽기를 마친 뒤에만 MuJoCo에서 실행합니다. 확인 포인트 — ㉠ 예측한 전이 순서대로 화면의 로봇이 움직이는가 ㉡ 팔 모션 중 하체·허리가 미세하게 움직이는가(균형 보상 — 4장 체크포인트 ㉢와 연결) ㉣ 모션 중 Ctrl+C를 실제로 눌러 미션 ②의 예측과 대조(시뮬레이션이므로 안전). 예측과 실측의 차이를 기록합니다.

미션 ④ 나만의 시퀀스 설계 (필수 · 종이에)

예제를 변형해 "기립 → 오른팔 인사 → 왼팔 인사 → 복귀" 같은 나만의 상체 모션 시퀀스를 종이에 설계합니다 — 사용할 명령, 순서, 각 대기 시간, 실패 시 경로까지. 아직 구현하지 않습니다: 7장 미션 과제 2의 초안이며, 목요일 설계일에 조에서 하나로 합칩니다.

순서	명령 (예제 기준 이름)	대기(초/조건)	기대 상태	실패 시 경로
①	Damp	-	힘 빠짐	-
②				
③				
④				
⑤	복귀/Damp		알려진 안전 상태	-

도전 미션 ★ 상태 확인의 자동화 (자율학습)

⑤의 리듬에서 "전이 완료 대기"를 고정 sleep이 아니라 상태 확인으로 바꾸면 어떻게 될까요? 3장에서 조사한 모드/상태 보고를 읽어 "기립 완료가 확인될 때까지 기다렸다가 다음으로" 진행하는 의사코드를 작성해 봅시다(구현은 선택). 고정 sleep 방식과 비교해 어떤 장단점이 있는지 세 줄로 — 시간 기반에서 상태 기반으로, 로봇 프로그래밍이 한 단계 성숙해지는 지점입니다.

```
# 힌트 - 상태 기반 대기의 빠대 (의사코드)
기립 명령 보내기
반복:
    m = 현재 모드 읽기          # 3장의 모드/상태 보고
    if m == 기립: break        # 확인되면 다음 단계로
    if 제한 시간 초과: Damp() 후 종료 # 실패 경로도 함께!
    잠시 대기
```

5.3 시뮬레이션 vs 실기체 — 같은 코드, 다른 긴장감

시뮬레이션에서 검증한 스크립트가 금요일 실기체에서 그대로 돌아가는 것이 이상적이지만, 현실에는 통신 지연·배터리 상태·바닥 조건 같은 변수가 있습니다. 시뮬레이션과 실기체의 차이(sim-to-real gap)를 직접 체감하는 것 자체가 이번 주의 중요한 학습이며, 연구실 연구 주제와 바로 맞닿아 있는 지점입니다.

무엇이 달라지는가 — 미리 알아두는 세 가지

변수	시뮬레이션에서는	실기체에서는
통신	내 컴퓨터 안 — 지연이 거의 0	네트워크를 거침 — 명령·상태에 지연과 흔들림
몸 상태	언제나 같은 이상적 모델	배터리 잔량·모터 온도에 따라 미묘하게 다른 거동
환경	완벽히 평평한 지면	바닥 재질·미세 요철 — 접촉 조건이 매번 다름

그래서 금요일의 자세는 "시뮬에서 됐으니 될 것"이 아니라 "시뮬에서 됐으니 시도할 자격이 생겼다"입니다. 같은 코드를 두 세계에서 돌려 보고 무엇이 달랐는지 한 줄이라도 기록하면, 여러분은 sim-to-real 연구의 첫 데이터를 만든 것입니다.

미션 ⑤ 금요일 실행 후보 선정 (필수 · 조 단위)

조원들이 각자 설계한 시퀀스(미션 ④) 중 금요일 오전 행어 위에서 실행할 후보 1~2개를 조에서 선정합니다. 선정 기준은 화려함이 아니라 — ㉠ 전이 경로가 모드 다이어그램 위에서 완결되는가 ㉡ 실패 경로가 설계되어 있는가 ㉢ 시뮬레이션에서 검증 가능한가. 선정안과 이유 세 줄을 조 채널에 남기고, 목요일 설계일에 최종 확정합니다 — 6.1절에서 보듯 학생 실행은 "멘토 확인 하"이므로, 멘토를 설득할 수 있는 설계가 곧 실행되는 설계입니다.

두 세계 비교 기록 양식 (수요일·금요일)

같은 스크립트를 시뮬레이션(수)과 실기체(금, 행어 위·멘토 확인 하)에서 실행한 뒤 아래 양식으로 대조합니다 — sim-to-real gap의 첫 관찰 데이터입니다.

관찰 항목	시뮬레이션에서	실기체에서 (금요일 기입)
전이에 걸린 체감 시간		
팔 모션 중 하체·허리의 보상		
예상과 달랐던 점 한 가지		

미션 ⑥ 금요일 리허설 대본 (필수 · 조 단위)

Go2 주간과 같은 형식으로, 금요일 오전 실행의 역할과 콜 순서를 미리 정합니다 — ㉣ 실행자(스크립트 실행) ㉤ 모니터링 담당(관절 상태 화면 주시, 6장의 핵심 임무) ㉥ 기록 담당(영상·학습일지). 비상정지는 이번 주 멘토 전담이므로 역할에서 제외됩니다. "상태 확인 콜 → 멘토 확인 → 실행 선언 → 실행 → 종료 상태 확인 콜"의 순서를 말로 한 번 리허설하고, 역할 로테이션 표를 조 채널에 남깁니다.

이번 장의 산출물 정리 5장을 마치면 남아 있어야 할 것 — ① 미션 ①의 모드 다이어그램(경로 색칠) ② 실패 경로 세 시나리오 답 ③ 시뮬레이션 검증 기록(예측 vs 실측, Ctrl+C 실험 포함) ④ 나만의 시퀀스 설계안(종이) ⑤ 도전 미션의 상태 기반 대기 의사코드(선택). ③·④가 금요일 오전(6장)과 미션 과제 2(7장)의 직접 재료입니다.

여기서부터 금요일까지 — 남은 길 지도 5장을 마친 지금, 남은 것은 실행뿐입니다 — 수요일: 미션 ③(시뮬 검증)과 4장 파라미터 실험 / 목요일: 미션 ④·⑤·⑥을 조에서 확정(설계일) / 금요일 오전: 6장 SOP에 따라 행어 위 실행 / 금요일 오후: 7장 미션. 각 단계의 산출물이 다음 단계의 입력이 되도록 repo에 쌓아 가세요.

5.4 자가 점검

다음 질문에 막힘없이 답할 수 있으면 6장(실기체 절차)으로 넘어갑니다.

- ① high-level 명령을 "상태 전이"로 이해한다는 것은 무슨 뜻인가? 모드 다이어그램을 그릴 수 있는가?
- ② "시작은 Damp, 이상 시 즉시 Damp"가 왜 원칙이며, StopMove 와 Damp 는 무엇이 다른가?
- ③ 코드 뼈대에서 Go2 대비 추가된 두 가지(사전 상태 확인·예외 처리)는 왜 필요한가?
- ④ 전이 사이의 대기를 고정 sleep 과 상태 확인으로 각각 구현하면 무엇이 다른가?
- ⑤ 시물과 실기체가 달라지는 세 변수(통신·몸 상태·환경)를 말할 수 있는가?
- ⑥ 상체 모션 시퀀스가 "기립(균형)" 상태에서만 의미가 있는 이유를 모드 다이어그램으로 설명할 수 있는가?
- ⑦ 금요일 실행 후보의 선정 기준 세 가지(㉠~㉢)를 말할 수 있는가?

다음 장 예고 코드가 준비되었으니 이제 실물 앞에 섭니다. 6장은 금요일 오전의 실기체 절차 — 멘토가 하는 것과 여러분이 하는 것의 경계, 표준 절차(SOP), 그리고 여러분의 핵심 임무인 관절 상태 모니터링을 다룹니다. 이번 주 안전 규정이 실제 동선이 되는 장입니다.

6. 실기체 절차 — 행어 위에서, 멘토와 함께

금요일 오전, 드디어 실물 G1 앞에 섭니다. 이 장의 목표는 두 가지입니다. 첫째, 멘토가 하는 것과 여러분이 하는 것의 경계를 정확히 알고 표준 절차(SOP)를 몸에 붙이는 것 — 이번 주 안전 규정이 실제 동선이 되는 순간입니다. 둘째, 실기체용으로 배부된 두 도구 — 모니터링 스테이션(g1_real_monitor.py)과 시퀀스 실행기(g1_real_sequence.py) — 를 여러분의 손과 목소리로 운전하는 것입니다. 5장까지의 시뮬레이션 도구가 g1edu 창구였다면, 이 둘은 실기체 창구(unitree_sdk2)의 도구입니다(3.1절의 두 창구가 오늘 실물이 됩니다). 이 장은 금요일 오전에 손에 들고 보는 장입니다 — 목요일에 한 번, 금요일 아침에 한 번 더 읽고 들어갑니다.

6.1 금요일 오전의 진행 방식

G1 실기체 시간은 멘토 주도 시연 + 학생 제한 조작으로 진행됩니다. 여러분이 하는 것과 멘토만 하는 것의 경계를 명확히 알고 시작합니다.

구분	내용
멘토가 주도	전원 투입·행어 거치·기립 전이·저속 보행 시연·이상 대응·모든 게이트의 최종 승인
학생이 수행 (멘토 확인 하)	리모컨 기초 체험(멘토 반보 뒤 동석) · 행어 거치 상태에서 상체 모션 스크립트 실행 · 관절 상태 모니터링 · 기록 · 파라미터 값 제한
항상 지정	비상정지 전담 1인 (리모컨 소지, 조작에 참여하지 않고 로봇만 주시) — 이번 주는 멘토가 담당

이 경계는 여러분을 배제하기 위한 것이 아니라, 35kg 휴머노이드의 실기체 시간에서 학생이 가장 많이 배울 수 있는 위치를 지정한 것입니다 — 코드는 여러분의 것(5장에서 검증한 그 스크립트), 실행 조건의 판단은 멘토의 것. "내 코드가 실물 위에서 돌아가는 것을 안전하게 보는 것"이 이번 주 실기체 경험의 핵심입니다.

행어 위에서도 진짜다

"행어에 매달린 로봇으로 뭘 배우나"라는 생각이 들 수 있습니다 — 반대입니다. 행어 위에서도 균형 제어는 실제로 돌고(2장에서 배운 발목·상체의 미세 조정이 눈앞에서 일어납니다), 상태 전이도, 여러분의 코드도, 통신도 전부 실물 그대로입니다. 행어가 없애는 것은 학습이 아니라 추락이라는 결과 하나뿐입니다. 오히려 행어 덕분에 실패해도 되는 실물 실험이 가능해지는 것 — 시뮬레이터가 소프트웨어의 안전망이라면, 행어는 하드웨어의 안전망입니다. 그래서 오늘 오전의 질문은 "언제 행어에서 내리나"가 아니라 "행어 위에서 얼마나 많이 관찰하고 기록하나"입니다.

오전 네 막의 흐름

오전은 네 막으로 진행됩니다. 각 막의 종료 상태가 다음 막의 시작 상태가 되도록 짜여 있으므로, 순서를

바꾸지 않습니다.

시간	활동	이 막이 끝나면
① 10:00 ~ 10:40	브리핑(3분) → 멘토 시연(리모컨 조합·제스처·저속 보행) → 학생 리모컨 체험(1인 5분 로테이션)	전원이 조합·콜을 몸에 익힘 · 로봇은 행어 하중 + 댐핑 대기
② 10:40 ~ 10:50	노트북 ↔ 기체 유선 연결 · IP 설정 · 연결 확인	인터페이스명 확정(화이트보드) · ping 응답
③ 10:50 ~ 11:05	g1_real_monitor: precheck → (재기립) → baseline → watch 가동 + 콜 훈련	기준선 커닝페이퍼 부착 · watch 상시 가동
④ 11:05 ~ 11:50	g1_real_sequence: 1회차 --arm-only → 2~4회차 전이 포함, 3역할 로테이션	전원이 실행자 경험 · 한 줄 로그 자동 누적
11:50 ~ 12:00	종료 절차(스트랩 하중 확인 → Damp) · 기록 정리 · 오후 브리핑	기록 5종 확인 · 오후 출발선

두 자리 — 실행석과 모니터링석

오늘의 공간에는 자리가 두 개 있습니다. **실행석**(시퀀스 실행기가 도는 노트북)과 **모니터링석**(모니터링 스테이션이 도는 노트북 + 커닝페이퍼·콜 카드). 두 노트북은 서로 통신하지 않습니다 — 같은 DDS 네트워크에 각자 붙어, 한쪽은 명령을 보내는 유일한 창구, 다른 쪽은 상태를 읽기만 하는 창구입니다(모니터 코드에는 명령 경로 자체가 없습니다). 두 화면을 잇는 유일한 배선은 사람의 목소리입니다 — 모니터링 화면이 띄운 콜을 담당이 소리로 외치고, 실행 화면의 게이트는 그 콜을 귀로 받은 뒤에만 넘어갑니다. 명령하는 눈과 감시하는 눈을 한 사람·한 프로세스에 겹치지 않는다 — 이것이 이 배치의 원칙이며, 노트북이 1대뿐이면 터미널 두 개(왼쪽 watch, 오른쪽 시퀀스)로 같은 원칙을 유지합니다.

자리는 고정, 사람이 순환 실행석·모니터링석·기록의 3역할은 사람이 로테이션합니다 — 자리와 화면은 그대로 두고 사람이 옮겨 앉습니다. 명령이 죽으면 관측으로 지켜보고, 관측이 죽으면 명령을 세운다 — 두 자리가 분리되어 있어야 이 비대칭이 성립합니다.

멘토 시연에서 무엇을 보는가 — 관람 과제

①막의 시연은 구경이 아니라 관찰입니다. 조원별로 하나씩 맡아 아래를 기록합니다 — 오후 미션과 회고의 재료입니다.

- **전이의 소리와 시간** : Damp→기립 전이에 실제로 몇 초가 걸리는가, 어떤 소리가 나는가 — 5장에서 정한 대기 시간과 비교
- **균형의 실물** : 기립 직후 발목·상체의 미세 움직임이 보이는가 — 시뮬레이터(4 장)·시물 데이터(3 장)에서 본 그 현상의 실물 확인

- **저속 보행의 발** : 발이 언제 어디를 딛는가, 한 발 지지 순간의 상체는 — 4.5 절 관찰의 눈을 실물에 적용
- **멘토의 손** : 멘토가 각 단계 사이에 무엇을 확인하고 무엇을 콜하는가 — 다음 학기에 여러분이 하게 될 동작의 연습

6.2 표준 절차 (Standard Operating Procedure)

SOP 여섯 단계는 그대로입니다 — 달라진 것은 그 절차의 앞뒤가 구체화되었다는 점입니다. 절차의 앞에는 기체가 깨어나는 순서와 리모컨이 있고, 절차의 한가운데에는 여러분의 스크립트가 게이트로 절차를 강제합니다. 기동부터 종료까지 순서대로 갑니다.

기체가 깨어나는 순서 — 전원·리모컨

No	단계 (멘토 수행, 여러분은 콜과 관찰)	확인
㉠	완전 현수 거치 : 양 어깨 서스펜션 버클에 스트랩, 발이 지면에서 떨어진 상태 · 관절 엉킴 없음	부팅은 완전 현수에서 — 발 접지는 기립 직전에
㉡	배터리 장착(전원 스위치가 등쪽) → 짧게 1회 + 2초 이상 길게	'딸깍' 체결음 · 억지로 누르지 않기
㉢	부팅 약 1분 — 전 관절 무토크(제로 토크)로 완료 판정	이 1분 동안 기체에 손대지 않기
㉣	리모컨 전원(짧게+2초 길게, 2회음) · 연결 확인	우측 DL 지시등 점등 = 기체와 연결
㉤	L2+B 댐핑 → L2+ ↑ 레디(중립 기립 자세) → 행어를 내려 발바닥 접지 → R2+A 균형 제어	레디는 완전 현수에서, R2+A는 접지 후
㉥	이후 운영 자세 : 발 접지 · 하중은 다리 · 스트랩은 느슨(추락만 방지)	스트랩이 팔·케이블에 걸리지 않게

리모컨 미연결 상태로 방치하면 기체가 10분 내 자동 종료되므로 세션 중 리모컨을 끄지 않습니다. 종료는 역순이되 한 가지 순서만 목숨처럼 지킵니다 — 행어를 올려 스트랩이 하중을 받는 것을 확인한 뒤에 댐핑(L2+B 또는 스크립트 Damp).

리모컨 기준표 — 오늘 쓰는 조합, 쓰지 않는 조합

오늘 쓰는 조합	입력	의미
비상 = 댐핑	L2+B	오늘의 유일한 비상 조작 — 로봇이 힘을 빼고 행어가 받는다
레디(위치잠금 기립)	L2+ ↑	중립 기립 자세로 잠김 (균형 제어는 아직 아님)
균형 제어(메인 컨트롤)	R2+A	능동 균형 시작 — 팔 액션·보행이 유효해지는 상태

오늘 쓰는 조합	입력	의미
제스처	SELECT 홀드 + Y / A / X	손 흔들기 / 악수 / 돌아보며 인사 (균형 제어에서만)
보행(멘토 시연)	더블클릭 L2 → START → 스틱	저속 모드 → 제자리 스텝↔정지 전환 → 좌 스틱 전후·우스틱 회전

오늘 누르지 않는 조합 L2+R2(디버그 — 모션 제어가 정지되는 SDK 개발용), L2+Y(제로 토크), L2+A(스쿼트), L2+X(눕기/기립), L2+←(착석). 그리고 학생 스크립트 실행 중 리모컨은 L2+B 전용입니다 — 제스처 조합을 함께 입력하면 코드의 명령과 충돌합니다. 조합은 펌웨어 버전에 따라 다를 수 있으므로(구버전은 L1+A/L1+↑/R1·R2+X 계열) 무반응 시 당황하지 말고 멘토에게: 펌웨어 차이 일 뿐입니다.

조이스틱 체험 프로토콜 (①막, 1인 약 5분)

리모컨은 주고받는 물건이 아니라 넘겨받는 권한입니다 — 반드시 "**리모컨 이양**" — "**리모컨 수령**" 콜과 함께 옮기고, 잡고 있는 사람이 그 순간의 비상정지 담당입니다(멘토는 손 뺄면 닿는 반보 뒤). 체험 코스는 제스처 1개(SELECT+Y 권장) → 희망자에 한해 저속 보행: START 스텝 → 전진 2보 → 스틱 중립(정지 확인) → 후진 2보 → 제자리 회전 반 바퀴 → START 정지 → 반납 콜. 규칙은 셋 — 한 번에 스틱 한 방향만, 손을 떼면 멈춘다, 이상하면 생각하지 말고 L2+B.

실기체 SDK 설치 — 목요일까지, 두 노트북 모두

②막이 순조로우려면 이 설치가 목요일에 끝나 있어야 합니다. 실기체 도구 둘

(g1_real_monitor.py·g1_real_sequence.py)은 unitree_sdk2py(제조사 공식 파이썬 SDK) 위에서 돕니다 — 시물 창구인 g1-edu 저장소와는 별개 설치이고, ROS와도 무관합니다: 3장에서 배운 DDS에 파이썬으로 직접 붙는 라이브러리입니다(그래서 실행·모니터링·식이 같은 상태 토픽을 나란히 구독할 수 있습니다). 실행 노트북과 모니터링 노트북 두 대 모두에 설치합니다. Python 3.8 이상이면 됩니다.

```
# 목요일까지 - 실행·모니터링 노트북 각각에서 한 번씩
sudo apt install python3-pip # 이미 있으면 그대로 통과
cd ~
git clone https://github.com/unitreerobotics/unitree_sdk2_python.git
cd unitree_sdk2_python
pip3 install -e .
python3 -c "import unitree_sdk2py; print('OK')" # OK가 찍히면 설치 완료
```

마지막 임포트 확인은 로봇 없이도 됩니다 — 로봇과의 연결 확인(ping)은 금요일 ②막의 일입니다. pip3 install -e . 단계에서 "Could not locate cyclonedds" 오류가 나면 의존 라이브러리(cyclonedds 0.10.2)를 소스 빌드해야 하는 환경입니다 — SDK README의 FAQ 절차(cyclonedds 0.10.x 빌드 → CYCLONEDDS_HOME 지정 → 재설치)를 따르되, 목요일 중 멘토와 함께 해결합니다. 금요일 아침에 발견하면 늦습니다. 배부 스크립트 2종은 조 repo에서 받아 아무 폴더(예: ~/g1_real/)에 두면 됩니다 —

unitree_sdk2py만 임포트되면 어디서든 됩니다.

연결 — 두 창구의 실물 (②막)

기체 목 뒤 이더넷 포트(가장 위쪽)에 랜선을 연결하고, 노트북의 유선 IP를 같은 대역으로 수동 설정합니다. DHCP는 쓰지 않습니다.

```
# 노트북 유선 IP: 192.168.123.222/24 (모니터링 노트북은 .223) - 수동 설정
ip addr                                # 인터페이스명 확인 → 이후 모든 --iface 값 (화이트보드에 기입)
ping 192.168.123.164                  # 기체 탑재 개발 PC(Jetson) - 응답이 오면 연결 완료
python3 -c "import unitree_sdk2py"    # 목요일 설치분(위 소제목) 최종 확인
```

DDS는 발행-구독 구조라서 상태(rt/lowstate)는 몇 대가 동시에 구독해도 서로 방해가 없습니다 — 실행 석과 모니터링석이 같은 은행의 다른 창구에 나란히 앉는 셈입니다(3.1절). ping이 3분 안에 안 잡히면 IP 대역 → 랜선·포트 → 허브 순으로 보고, 그래도 안 되면 노트북 1대·터미널 2개 구성으로 전환해 시간을 지킵니다.

SOP 6단계 — 이번 주 배운 모든 것의 실행 순서

여섯 단계를 순서대로 — 각 단계가 끝났음을 소리 내어 확인(콜)하고 다음으로 넘어갑니다. 항공기 조종사들의 체크리스트 문화와 같은 방식입니다.

No	절차	확인 콜(예) · 담당
①	조작 전 5단계 체크 (Week 1 안전교육의 체크리스트 — 예약표·동석·비상정지·공간·코드 설명)	"5단계 체크 완료" · 조 전원
②	행어 거치 확인 : 스트랩·자세를 멘토가 확인하고 시작	"행어 거치 확인" · 멘토
③	전원 투입 → Damp 상태 확인 → precheck로 이상값 없는지 확인	"Damp 확인, 상태 정상" · 멘토+모니터링
④	행어 거치 상태에서 기립 전이 → 상체 모션 실행 (학생 차례는 여기부터)	"기립 완료 — 모션 실행합니다" · 실행자
⑤	저속 보행은 멘토 주도로만 : 행어 추종 또는 지정 구역, 반경 2m 확보, 전원 위치 확인 후	"반경 확보, 전원 위치 확인" · 멘토
⑥	종료 : StopMove → 스트랩 하중 확인 → Damp → 행어 정위치 → 전원 차단 → 예약표 기록	"종료 절차 완료" · 멘토+기록

이 표를 이번 주 내내 쌓아 온 개념으로 다시 읽어 보세요 — ①은 교재 ① 안전교육, ②는 2장의 행어 원리, ③은 3장의 상태 확인 루틴, ④는 5장의 상태 전이와 여러분의 코드, ⑤는 2.2절의 반경 2m 물리, ⑥은 "끝도 알려진 상태로"(5장 코드 뼈대의 ⑥)입니다. SOP는 외울 목록이 아니라, 이번 주 배운 모든 것의 실행 순서입니다.

스크립트가 절차를 강제한다 — 게이트와 콜의 1:1

④단계의 실행기(g1_real_sequence.py, 6.4절이 아니라 아래에서 자세히)는 SOP를 화면의 게이트로 옮겨 놓았습니다 — y를 입력해야만 다음으로 넘어가고, y 외의 입력은 전부 Damp 수렴 경로입니다. 그래서 콜 대본은 외우는 대사가 아니라, 화면이 순서대로 물어보는 질문에 목소리로 답하는 것입니다.

```

실행자 : "5단계 체크?"          조 전원 : "완료"          [게이트 y]
멘토   : "행어 거치 확인"        [게이트 y]
실행자 : "멘토 리모컨 확인 부탁드립니다"  멘토   : "리모컨 확인" [게이트 y]
실행자 : "모니터링 가동?"        모니터링: "가동 중"    [게이트 y]
멘토   : "Damp 승인" → 스크립트 Damp 전송
모니터링 : "상태 정상"          [게이트 y] ← SOP ③
멘토   : "기립 승인" → 전이 → 실행자 육안 확인 y
멘토   : "균형 제어 승인" → 전이 → 육안 확인 y
실행자 : "기립 완료 - 모션 실행합니다" ← SOP ④ (스크립트가 대신 출력)
...(동작 중 이상 시) 모니터링 콜 → 실행자 손 멈춤 → 멘토 판단
실행자 : "시퀀스 완료" → 종료 선택(k=기립 유지 / d=Damp·멘토 승인)

```

전이 화면 읽는 법 — 번호가 달라도 당황하지 않기

전이 중 화면에는 FSM이라는 상태 번호가 흐릅니다 — 댐핑 1, 기립 4(또는 706), 균형 제어 500(구버전 200). 이것은 2.4절 상태 기계의 실기체판 번호일 뿐입니다. 기대 번호가 15초 안에 안 잡히면 스크립트는 "육안으로 판정"으로 강등합니다 — 예러가 아니라 펌웨어별 차이이며, 육안 확인 게이트가 언제나 최종 판정입니다. 관측된 번호는 화이트보드에 적어 다음 조에 인수인계합니다(그 조의 사전 검증 시간이 절반이 됩니다).

절차에서 벗어나는 유일한 방법 어느 단계에서든 이상(이상 소음·경고·비정상 자세·모니터링 담당의 콜)이 감지되면 — 진행 중이던 것을 멈추고 멘토의 판단을 기다립니다. "조금 이상하지만 계속"은 이번 주에 존재하지 않는 선택지입니다. 이상 보고는 실습을 방해하는 것이 아니라 실습의 일부입니다 — 잘못 보고해도 칭찬받는 것이 우리 연구실의 규칙입니다.

이상 상황 미리 그려보기 — 무엇이 일어날 수 있고, 그때 순서는

상황	대응 순서 (판단은 멘토, 우리는 멈춤과 보고)
기립 전이가 중간에 멈춤·실패	실행자 손 멈춤 → 모니터링 콜 → 멘토가 Damp·재시도 판단 — 학생이 재시도 명령을 먼저 보내지 않는다
모션 중 이상 소음·진동	"이상 소음" 콜 → 실행 중단 → 멘토 점검 — 소리는 데이터에 안 찍히는 중요한 센서입니다
스크립트 응답 없음(멈춘 듯)	Ctrl+C를 바로 누르지 말고 먼저 콜 — 멘토 지시로 중단(중단돼도 스크립트는 Damp로 수렴하도록 만들어져 있음)
통신 끊김(상태 데이터 정지)	"모니터링 끊김" 콜 → 전원·조작 모두 대기 — 보이지 않는 로봇에게 명령하지 않는다
팔 액션이 거부됨(화면에 code≠0)	십중팔구 상태 문제 — 균형 제어 진입 전이거나 직전 동작 직후의 연타. 2.4절 상태 기계로 돌아가 확인, 임의 재전송 금지

상황	대응 순서 (판단은 멘토, 우리는 멈춤과 보고)
낙하·스트랩에 하중 발생	"행어 하중" 콜 → 전원 접근 금지 → 멘토만 접근해 Damp 확인 후 제거치

오늘 유일한 고위험 실수 스트랩 하중 확인 전의 댐핑. 댐핑은 로봇이 힘을 빼는 명령이므로, 스트랩이 받을 준비가 되어 있지 않으면 그대로 주저앉는 사고입니다. 종료·전환의 모든 댐핑은 "행어를 올려 스트랩이 하중을 받는 것을 확인 → 그 다음 L2+B(또는 승인)" 순서를 지킵니다.

6.3 관절 상태 모니터링 — 학생의 핵심 임무

조작하지 않는 조원은 노는 시간이 아닙니다. 모니터링 담당은 "지금 로봇이 정상인가"를 계속 읽는 사람입니다. 지난주까지는 r

os2 topic echo로 흘러 보았다면, 오늘은 그 일을 전담하는 도구가 배부됩니다 — 6.5절 도전 미션("모니터링 화면 설계")의 완성형이 먼저 도착한 셈이니, 쓰면서 "나라면 무엇을 더 보이게 할까"를 생각해 보세요. 이 도구는 읽기 전용입니다: 로봇에 어떤 명령도 보내지 않으므로, 언제 켜 두어도 안전합니다.

```
python3 g1_real_monitor.py precheck --iface <이름> # ③단계 - "상태 정상" 콜의 근거
python3 g1_real_monitor.py baseline --iface <이름> --sec 15 # 기준선 캡처 (균형 기립 상태에서!)
python3 g1_real_monitor.py watch --iface <이름> # 실시간 감시 - 콜 문구를 그대로 띄움
```

모드	언제	화면에 뜨는 것	여러분이 하는 것
precheck	SOP ③, 각 사이클의 Damp 직후	수신 Hz·tick·IMU 기울기·최고 온도·최대 토크·NaN의 PASS/CHECK 표 + 콜 제안	표를 근거로 "상태 정상" 또는 "점검 필요 — CHECK 항목 보고"를 외침
baseline	균형 제어 진입 직후 1회 (하루 한 번이면 충분)	15초 캡처 → 기준선 파일 저장 + 커닝페이퍼(토크 상위 관절의 평소 상한·IMU 평소값)	커닝페이퍼를 모니터링석에 부착
watch	학생 차례 시작부터 끝까지 상시	1초마다 관측 한 줄(기울기·발목 q/tau) · 이상 시 벨과 함께 콜 문구 · CSV 자동 적재	화면의 콜 문구를 소리로 옮김 — 작문하지 않는다

무엇이 "정상"인가 — 기준선을 먼저 잡는다

"비정상적으로 크다"를 판정하려면 정상값을 알아야 합니다. 그래서 모니터링의 첫 임무는 기준선을 잡는 것 — 그리고 어느 상태에서 잡느냐가 판정의 의미를 결정합니다. 오늘의 기준선은 균형 제어(기립) 상태에서 잡습니다: 댐핑 상태에서 잡으면 전 관절이 0 근처라, 기립 전이의 정상 토크가 전부 경보로 쏟아지기 때문입니다. 기립 유지 토크가 "평소"가 되면, 댐핑·기립·팔 동작 전 구간에서 진짜 이상만 걸립니다. 3장 미션 ②·도전 미션에서 시뮬레이터로 해 본 "관절 하나 추적"이 정확히 이 훈련이었습니다 — 시물에서

본 기준과 실물의 기준이 어떻게 다른지도 좋은 기록거리입니다.

무엇을 외치는가 — 모니터링 콜 규약

상황	콜(예) — 짧고, 크게, 즉시
정상 확인 요청을 받았을 때	"상태 정상" (③·④ 단계 진입 전 확인 콜 — precheck의 PASS 표가 근거)
토크가 기준선 대비 눈에 띄게 큼	"토크 이상 — ○○ 관절" → 실행자는 즉시 손 멈춤, 멘토 판단
IMU 기울기가 서서히 증가	"기울기 증가 중" → 멘토 판단 (행어 위라도 보고)
모터 온도 경고	"온도 주의 — ○○ 관절" → 멘토가 계속/휴식/정리 판단
화면이 멈추거나 데이터 끊김	"모니터링 끊김" → 데이터 없이 진행하지 않는다 — 복구까지 대기

콜은 문장이 아니라 신호입니다 — 상황·위치만 짧게, 판단은 멘토에게. watch는 이 문구들을 벨 소리와 함께 화면에 그대로 띄우므로, 담당의 일은 작문이 아니라 낭독입니다. "확실하지 않으면 외친다"가 기본값이며, 오탐(false alarm)은 비용이 거의 없지만 미탐(놓침)은 사고입니다 — 기립 전이의 과도 토크가 임계를 스쳐 콜이 한 번 뜨는 것은 값싼 오탐으로 처리하면 됩니다.

모니터링 담당의 화면 배치와 시선

모니터링은 두 화면을 동시에 봅니다 — 한쪽엔 watch, 다른 쪽엔 로봇 실물. 화면만 보면 실물의 이상 징후(자세·소리)를 놓치고, 실물만 보면 숫자의 이상을 놓칩니다. 요령은 "숫자 10초 → 실물 5초"처럼 시선을 규칙적으로 오가는 것, 그리고 실행 콜 직후에는 반드시 숫자를 보고 있는 것입니다(명령 직후가 이상값이 나타나는 순간이므로). 노트북 위치는 로봇과 화면이 한 시야에 들어오는 자리로, 볼륨은 커 둡니다(경보 벨). ③막에서 끊김 드릴을 한 번 합니다 — 랜선을 2초 뽑았다 꽂아 "모니터링 끊김" 콜을 실제로 외쳐 보는 것. 로봇에는 아무 영향이 없는, 가장 안전한 콜 훈련입니다.

역할 로테이션 조작 / 비상정지 전담 / 모니터링·기록의 3역할을 페어로 돌아가며 맡습니다. 전원이 세 역할을 모두 경험하는 것이 목표입니다. (이번 주 비상정지는 멘토 담당이므로, 학생 로테이션은 실행자 ↔ 모니터링 ↔ 기록의 3역할로 운영합니다 — 5장 미션 ⑥의 로테이션 표대로)

6.4 실행과 기록 — 시퀀스 실행기, 그리고 남기는 법

④막의 주인공은 실행자와 시퀀스 실행기입니다. 실행기는 여러분이 7장 과제 2에서 설계할 SEQUENCE(상체 동작 행)를 받아, 전이(Damp→기립→균형)와 종료를 멘토 게이트와 함께 수행합니다 — 5장에서 시뮬레이터로 검증한 그 흐름의 실기체판입니다.

실행자가 겪는 한 사이클 (약 3~5분)

구간	무엇이 일어나는가
계획 표	실행 직후 SEQUENCE가 실행 계획 표로 출력 — 설계 시트와 행 단위 대조(기

구간	무엇이 일어나는가
	록 담당. 금지 행(move 등)-미지원 동작(bow)은 이 시점에 걸러지고 로봇에 아무것도 보내지 않은 채 종료
게이트 4개	5단계 체크 / 행어 거치 / 멘토 리모컨 / 모니터링 가동 — 콜을 귀로 받은 뒤에만 y
전이 3개	Damp(직전이 기립이었다면 스트랩 쪽으로 내려앉는 것이 정상) → "상태 정상" 콜 게이트 → 기립 → 균형 제어. 각 전이는 멘토 승인 + FSM 확인 + 육안 확인
동작 행	각 동작 후 고정 대기(카운트다운) — 실기체 상위 명령에는 시물의 완료 신호(ActionActive)가 없기 때문. 이 차이 자체가 sim-to-real 관찰 항목
종료 선택	k = 기립 유지(다음 실행자 인계, 권장) / d = Damp(멘토 승인·스트랩 하중 확인 콜). 마지막 회차만 d
이상·중단 시	어느 지점에서든 결말은 하나 — Damp 수렴 + 사유가 로그에 기록. Ctrl+C·게이트 불통과·명령 거부 전부 같은 경로

첫 회차는 --arm-only(멘토가 리모컨으로 균형 제어까지 올려 둔 상태에서 팔 동작만)로 감을 잡고, 2회차부터 전이 포함 전체를 돕니다. 회차 사이 교대는 90초 — 역할이 한 칸씩 돌고, 다음 실행자는 자기 이름으로 재실행합니다.

기록 담당의 임무 — 오후와 다음 조를 위해

세 번째 역할, 기록 담당의 산출물이 오후 미션과 다음 조의 출발선을 만듭니다. 세 가지를 남깁니다.

- **영상** : 학생 차례의 상체 모션 실행 장면 — 5.3 절 비교 양식의 "실기체에서" 열을 채울 근거이자 회고의 재료
- **데이터** : watch 가 자동으로 쌓는 모니터링 CSV(10Hz — bag 의 대응)와, 실행기가 자동 기록하는 한 줄 로그 CSV. unitree_ros2 가 떠 있으면 bag 녹화를 병행 — "로봇 시간은 아껴 쓰고 데이터는 남긴다"(Go2 주간과 동일)
- **한 줄 로그 확인** : 실행기가 시각/동작/실행자/결과/이상 콜/소요를 자동 기록하므로, 담당의 일은 작성이 아니라 확인 — 각 회차 종료 때 마지막 행이 실제와 맞는지 대조하고, 종이 예약표를 병행합니다

이번 세션의 산출물 정리 오전이 끝나면 남아 있어야 할 것 — ① 관람 과제 4항목의 기록 ② 학생 실행의 영상·CSV 2종(모니터링·한 줄 로그) ③ 5.3절 비교 양식의 "실기체" 열 ④ 예약표 기재 완료. 이 기록들이 오후 미션(7장)의 출발 자료이자 인수인계 노트의 재료입니다.

한 줄 로그 양식 (자동 기록과 대조용)

시각	스크립트/동작	결과 한 줄	이상 콜	담당
(예) 11:12	g1_real_sequence [wave→hands_up]	정상 완료 (3행), 187초	무	○○○

6.5 자율학습 미션 — 금요일 오전 준비 (목요일까지)

미션 ① 콜 대본 리허설 (필수 · 조 단위)

6.2절의 콜 대본을 조에서 역할대로 소리 내어 한 번 통과합니다(로봇 없이, 콜만 — 게이트 y까지 포함해서). 콜이 어색하지 않을 때까지 — 금요일에 처음 말해 보는 콜은 반드시 늦습니다.

미션 ② 기준선 연습 (필수)

시뮬레이터에서 기립 상태의 low state를 30초 관찰하며 "내가 기준선으로 삼을 관절 3개와 그 평소 값 범위"를 정해 학습일지에 적습니다 — 금요일 모니터링 담당이 되었을 때 baseline의 커닝페이퍼와 나란히 붙일 나만의 연습장입니다. 왜 기준선을 균형 기립에서 잡는지(6.3절)를 한 문장으로 함께 적으면 완성.

미션 ③ 콜 규약 암기 확인 (필수 · 페어)

조원과 짝지어 6.3절의 다섯 상황을 무작위로 불러 주고 콜을 즉답하는 연습을 합니다 — 5초 안에 나오지 않으면 다시. 안전 콜은 검색하는 것이 아니라 반사적으로 나와야 합니다.

미션 ④ 금요일 아침 최종 점검표 (필수 · 조 단위)

목요일 밤, 조 채널에 금요일 아침 점검표를 만들어 둡니다 — ㉠ 실행할 시퀀스 최종본이 조 repo에 있고 시물 검증 기록이 붙어 있는가 ㉡ 역할 로테이션 표 확정 ㉢ 조원 전원이 콜 대본·모니터링 콜 리허설 완료 ㉣ 기준선 연습장(미션 ②) 준비 ㉤ 두 노트북 모두 unitree_sdk2py импорт 확인(6.2절 설치)·유선 IP 설정(192.168.123.x)·충전·녹화 공간 확보. 금요일 아침 10분 만에 이 다섯 항목에 체크만 하면 되는 상태가 목표입니다.

미션 ⑤ 이상 상황 카드 훈련 (필수 · 페어)

6.2절의 이상 상황 표 여섯 가지를 카드처럼 무작위로 뽑아, 한 명이 상황을 부르면 다른 한 명이 "누가 무엇을 멈추고, 무슨 콜을 하고, 판단은 누구에게 넘기는가"를 10초 안에 답합니다. 콜 규약 훈련(미션 ③)이 단어 연습이라면 이것은 문장 연습입니다 — 상황과 대응이 짝으로 나와야 금요일에 몸이 움직입니다. 여섯 상황 모두 양방향으로 한 번씩 통과하면 완료.

도전 미션 ★ 모니터링 도구 해부 (자율학습)

배부된 g1_real_monitor.py를 열어 세 가지 질문에 답해 봅니다 — ① 토크 경보의 임계는 어떻게 계산되는가(기준선의 몇 배, 바닥값은 얼마) ② "서서히 증가"는 몇 초 창으로 판정하는가 ③ 내가 담당이라면 화

면에 무엇을 하나 더 띄우고 싶은가. ③의 제안이 좋으면 다음 조에게 물려줄 도구 개선의 씨앗이 됩니다 — 도구는 쓰는 것에서 읽는 것으로, 읽는 것에서 고치는 것으로 자랍니다.

다음 조에게 — 오전 세션의 인수인계 이번 주 실기체 오전을 가장 크게 개선하는 것은 앞 조의 기록입니다. 인수인계 노트(조 repo 최상단)에 세 줄을 남깁니다 — ① 우리 조의 기준선 값(어느 관절, 평소 범위)과 전이 체감 시간·관측 FSM 번호 ② 우리를 가장 오래 붙잡은 문제 하나와 해결책 ③ "금요일 아침에 이것부터 확인하세요" 한 줄. Go2 주간의 HANDOVER.md와 같은 형식 — 받은 조는 그 가치를 이미 알고 있습니다.

6.6 자가 점검

다음 질문에 막힘없이 답할 수 있으면 7장(금요일 미션)으로 넘어갑니다.

- ① 멘토가 주도하는 것과 학생이 수행하는 것의 경계를 말할 수 있는가?
- ② SOP 여섯 단계를 순서대로, 각 단계의 확인 콜과 함께 말할 수 있는가?
- ③ 리모컨의 "오늘 쓰는 조합" 다섯 가지와 비상 조작 하나를 즉답할 수 있는가?
- ④ 실행석과 모니터링석을 나누는 이유 — 두 자리를 잇는 유일한 배선이 무엇인지 말할 수 있는가?
- ⑤ 기준선을 어느 상태에서 잡는지, 그 이유를 설명할 수 있는가?
- ⑥ 다섯 가지 모니터링 콜을 5 초 안에 말할 수 있는가?
- ⑦ "오탐은 싸고 미탐은 비싸다"를 안전 문화의 언어로 설명할 수 있는가?
- ⑧ "하중 확인 전 댐핑 금지"가 왜 오늘 유일한 고위험 실수인지 물리로 설명할 수 있는가?
- ⑨ 행어가 없애는 것은 학습이 아니라 추락이라는 말을 자신의 언어로 설명할 수 있는가?

다음 장 예고 오전에 절차를 완주했으니, 오후에는 이번 주의 결승선 — 보행 파라미터 튜닝 + 상체 모션 시퀀스 미션입니다. 4장의 관찰표와 5장의 시퀀스 설계가 드디어 하나의 미션으로 합쳐지고, 판정과 실행을 도와줄 도구 두 개가 배부됩니다.

7. 금요일 미션 — 보행 파라미터 튜닝 + 상체 모션 시퀀스

이 장은 한 주의 수렴점입니다. 2장에서 균형의 원리를, 3장에서 데이터를 조사하는 법을, 4장에서 MuJoCo 관찰표를, 5장에서 API와 상태 전이를, 6장에서 행어 위의 표준 절차를 익혔습니다 — 그 전부가 이 미션의 준비였습니다. 미션은 산출물 두 개를 요구합니다. 하나는 시뮬레이션에서, 하나는 행어 거치 실기체에서 만듭니다. 두 무대를 오가는 이 구조 자체가 연구실이 매일 도는 사이클 — 시뮬레이션에서 검증하고 실기체로 옮기는 *sim-to-real* — 의 축소판입니다. 오늘은 그 사이클을 도와줄 도구 둘이 함께 배부됩니다: 과제 1의 반복 판정을 자동화하는 `mission7_judge.py`, 그리고 오전에 이미 만난 실기체 실행기 `g1_real_sequence.py`. 이 장의 목표는 두 과제를 "해봤다"가 아니라 "근거를 갖고 통과했다"로 마치는 것입니다.

7.1 미션 정의

오후(14~18시)에 조별로 두 가지 산출물을 만듭니다. 하나는 시뮬레이션에서, 하나는 행어 거치 실기체에서 완성합니다.

항목	내용
과제 1 (시뮬레이션)	보행 파라미터 튜닝 : 수요일 관찰표를 바탕으로 "가장 안정적인 보행"과 "가장 빠르되 넘어지지 않는 보행"의 파라미터 세트를 각각 찾아 근거와 함께 제시 — 판정은 <code>mission7_judge.py</code> 로 반복 실행
과제 2 (실기체·행어)	상체 모션 시퀀스 : 5장 예제를 변형해 2~3개 동작을 잇는 나만의 시퀀스 제작 → 시뮬선검증 → 코드 리뷰 → 행어 거치 상태에서 멘토 확인 하 실행(<code>g1_real_sequence.py</code>)
도전 과제	MuJoCo에서 보행 + 상체 모션 동시 실행 — 균형에 어떤 영향이 있는지 분석 (시뮬레이션 한정)
제출물	파라미터 비교표 + 시퀀스 코드 + 실행 영상 + 한 페이지 회고 → 조 repo (금요일 오후 내)

왜 무대가 두 개인가 — 배정의 원리

보행 파라미터는 넘어질 수 있는 실험이므로 시뮬레이션이 무대이고, 상체 모션은 행어가 지켜주는 실험이므로 실기체가 무대입니다. "위험한 것은 시뮬레이션에서, 안전이 확보된 것만 실기체에서" — 이 배정 원리를 이해하면, 앞으로 어떤 실험을 만나도 무대를 스스로 고를 수 있게 됩니다. 두 과제가 요구하는 역량도 다릅니다.

구분	과제 1 · 보행 파라미터 튜닝	과제 2 · 상체 모션 시퀀스
재료	수요일 관찰표 (4장)	미션 B 예제 코드 (5장)
핵심 역량	변인 통제 · 근거 기반 주장	상태 전이 설계 · 안전 절차 준수

구분	과제 1 · 보행 파라미터 튜닝	과제 2 · 상체 모션 시퀀스
도구	param_sweep.py(경계 찾기) · mission7_judge.py(확정 세트 판정)	mission4_sequence.py(시물 선검증) · g1_real_sequence.py(실기체 실행)
연결 장	2장(균형) · 4장(관찰표)	5장(API) · 6장(SOP)
진행 단위	조 공동 1건 (운전은 로테이션)	개인 1건씩 — 각자 자기 시퀀스로 행어 슬롯 1회

통과의 정의 — 시작 전에 합의하기

Go2 주간과 마찬가지로, 미션은 시작 전에 "통과"를 판정 가능한 문장으로 조에서 합의하고 시작합니다. 애매한 기준은 애매한 회고를 낳습니다. 아래는 최소 기준이며, 조에서 더 구체화해 학습일지 첫머리에 적습니다.

과제	통과 기준 (최소)
과제 1	두 세트 각각에 대해 같은 조건 반복 실행에서 연속 완주(예: 3회 중 3회 넘어지지 않음)를 확인하고, 세트 간 차이를 관찰표의 행을 근거로 한 문장으로 설명할 수 있다
과제 2	동작 2개 이상을 잇는 시퀀스가 행어 거치 상태에서 시작(Damp)부터 복귀(Damp)까지 멈춤 없이 완주하고, 실행 전 코드 설명(실행 전 설명 가능 원칙)을 멘토에게 통과했다
공통	제출물 4종(7.6절)이 금요일 오후 내 조 repo에 올라가 있다

"넘어지지 않는"의 정의도 합의 대상 속달 세트의 "넘어지지 않는"은 사람마다 다르게 읽힙니다. 몇 회 중 몇 회 완주를 기준으로 할지, 시도당 실행 시간은 얼마로 할지 시작 전에 숫자로 합의하세요 — 그 숫자가 그대로 판정 도구의 반복 횟수·실행 시간 설정이 됩니다. 기준을 세우는 일 자체가 실험 설계 훈련입니다.

한 주 개념 회수표 — 배운 것이 어디서 쓰이는가

미션 중 막히면 이 표의 오른쪽 열이 "어느 장으로 돌아갈지"의 지도입니다.

배운 곳	개념	미션에서 쓰이는 곳
2장	지지 영역 · 자세 보상 vs 스텝 · 무게중심	과제 1의 트레이드오프 해석, 도전 과제의 균형 분석
3장	상태 토픽 조사법 (데이터를 눈으로 확인)	행어 실행 중 관절 상태 모니터링의 독법
4장	관찰표 · 한 번에 하나 · 이분 탐색	과제 1의 근거 자료와 세트 설계
5장	상태 전이 · Damp 원칙 · 미션 B 예제	과제 2의 시퀀스 설계와 코드 리뷰
6장	SOP 6단계 · 게이트와 콜 · 3역할과 두 자리	행어 실행의 전 과정 — 순서를 건너뛰지 않기

7.2 목요일을 미션 설계일로 쓰기

목요일에 조원과 함께 ① 수요일 관찰표를 정리해 튜닝 후보 파라미터를 2~3개로 좁히고, ② 상체 모션 시퀀스의 동작 순서와 전이 대기 시간을 설계하고(각자 1건), ③ 금요일 3역할 로테이션과 행어 슬롯 순서를 정해 두면, 금요일 오후를 온전히 실험에 쓸 수 있습니다.

설계일이 잘 쓰였는지는 금요일 아침에 판정할 수 있습니다 — 아래 여섯 줄이 모두 "예"면 준비된 것입니다.

- 통과 기준(7.1)이 판정 가능한 문장으로 학습일지 첫머리에 적혀 있는가
- 수요일 관찰표가 한 파일로 합쳐져 조원 전원이 볼 수 있는 상태인가 — 후보 파라미터 2~3 개에 ○ 표시까지
- 두 세트의 초안(값 + 근거 행 번호)이 잡혀 있는가 — 금요일에는 판정 도구에 옮겨 적기만 하면 되도록
- 시퀀스 설계 시트(7.3)가 각자 채워져 있고, 상태 전이 그림을 담당자가 안 보고 그릴 수 있는가
- 행어 슬롯 순서와 3 역할 로테이션이 정해져 있는가 — 비상정지 전담(멘토)의 리모컨 위치까지 확인
- 제출물 4 중(7.6)의 담당자가 나뉘어 있는가 — 정리를 후반에 몰아서 하면 반 3 드시 빠뜨립니다

7.3 접근 전략 — 단계적으로

- **단계 1** : 기본 파라미터로 시뮬레이션 보행 재현 (수요일 상태로 복귀)
- **단계 2** : 파라미터를 한 번에 하나씩만 변경 — 두 개를 동시에 바꾸면 원인을 알 수 없습니다 (변인 통제!)
- **단계 3** : 시퀀스는 동작 2 개부터 — Damp 복귀까지 완주가 확인된 뒤에 동작을 추가
- **단계 4 (도전)** : 동시 실행은 시뮬레이션에서만 — 실기체 반영 여부는 멘토가 판단합니다

설계 힌트 "가장 안정적"과 "가장 빠른"은 대개 반대 방향의 파라미터입니다. 두 세트의 차이를 표로 대비시키면, 속도와 안정성의 트레이드오프라는 보행 제어의 핵심 개념이 스스로 드러납니다 — 회고에 이 관찰을 적으면 훌륭한 결과물입니다.

과제 1 풀어가기 — 관찰에서 판정으로

반복 실행이라는 지루한 노동은 판정 도구가 맡습니다. 여러분의 몫은 세트를 근거로 설계하는 것과, 결과를 해석하는 것 — 도구가 일부러 비워 두는 두 칸입니다.

```
# 저장소 루트(~/robot_ws/src/g1-edu)에서
python3 param_sweep.py --set step_period=0.40 --vx 0.35 # 경계를 모르는 값만 사전 확인
python3 mission7_judge.py --dry-run # 계획 낭독 - 통과 기준과 대조
python3 mission7_judge.py # 본 판정: 두 세트 x 3회 x 30초
```

(헤드리스)

```
python3 mission7_judge.py --only fast --view --repeats 1 # 대표 1회 뷰어 실행 - 영상 녹화용
```

- ① **관찰표 복사** : 목요일에 줬힌 후보 파라미터 2~3 개를 중심으로, "X 를 Y 로 바꾸면 Z 가 일어난다"는 행들을 오늘의 근거 자료로 씁니다.
- ② **후보 세트 설계** : 안정 세트는 보수적인 값(낮은 속도·보폭)에서, 속달 세트는 관찰표에서 찾은 안전 경계 바로 안쪽에서 시작합니다. 경계를 모르는 파라미터는 param_sweep.py 로 경계부터 찾습니다(4 장의 이분 탐색 그대로) — 경계 밖 값은 세트로 제출하지 않습니다.
- ③ **세트 기입과 낭독** : mission7_judge.py 상단의 학생 편집 구역(SETS)에 두 세트를 값과 근거(관찰표 행 번호)까지 적고, --dry-run 출력을 조끼리 소리 내어 읽으며 통과 기준과 대조합니다. 한 세트에서 두 파라미터를 동시에 바꿨다면 여기서 걸립니다(변인 통제).
- ④ **판정 실행** : 각 세트를 같은 조건(매번 새 시뮬레이터·같은 시작 자세·같은 실행 시간)으로 반복 실행합니다 — 도구가 자동으로 보장합니다. 회차마다 완주/넘어짐·낙상 시각·실측 평균 속도·최대 기울기가 기록되고, 끝에 "3/3 완주 — 통과" 판정이 붙습니다.
- ⑤ **대표 영상** : 통과 세트로 --view 1 회를 화면 녹화 — 제출물 ③의 절반입니다.
- ⑥ **비교표 완성** : 자동 생성된 비교표 파일의 근거 열과 한 줄 해석은 비어 있습니다 — 그 두 칸을 채우는 것이 과제의 본체입니다.

항목	① 안정 세트	② 속달 세트	근거 (관찰표 행 번호)
전진 속도 vx	(값)	(값)	예: 관찰표 3행
회전 속도 vyaw	(값)	(값)	
보폭 / 주기 등	(값)	(값)	
반복 실행 결과	예: 3/3 완주	예: 3/3 완주	실행 영상 타임스탬프
실측 평균 속도	(자동 기록)	(자동 기록)	명령값과 다름을 관찰
한 줄 해석	두 세트의 차이가 말해주는 것 — "속도를 얻는 대가로 무엇을 지불했는가"를 2장의 언어(지지 영역·스텝 타이밍)로 씁니다		

실측 속도가 명령값보다 작게 나옵니다 고장이 아니라 저장소 README에 적힌 정직한 특성입니다 — 명령과 실측이 1:1이 아닌 이유를 스스로 추측해 회고에 적으면, 그 자체가 좋은 관찰 산출물입니다.

과제 2 풀어가기 — 종이에서 행어까지, 같은 세 열이 세 번

여러분의 시퀀스는 세 번 옮겨 적히며 무대를 바꿉니다 — 설계 시트(종이) → 시물

SEQUENCE(mission4_sequence.py) → 실기체 SEQUENCE(g1_real_sequence.py의 개인 사본). 세 곳 모두 같은 세 열(동작·인자·대기 시간)이므로, 옮겨 적기는 번역이 아니라 이식입니다. 단, 실기체 사본에는 상체 동작 행만 옮깁니다 — 전이(Damp·기립·균형)와 종료는 멘토 게이트가 달린 프레임의 소관이고, 보행 행(move)은 코드가 거부합니다(SOP ⑤).

- ① **설계(종이)** : 아래 시퀀스 설계 시트를 먼저 채웁니다. 상태 전이 순서(Damp 에서 시작해 어디를 거쳐 Damp 로 돌아오는가)를 그림으로 그리고, 전이 사이의 대기 시간과 "실패하면 어디로 복귀하는가"를 동작마다 적습니다 — 목요일 설계일의 산출물입니다. bow 처럼 시물 전용 동작은 이 단계에서 걸러 둡니다(실기체 지원 동작 목록은 멘토가 사전 검증에서 확정).
- ② **시뮬레이션 선검증** : mission4_sequence.py 에 옮겨 전이 순서·대기 시간이 성립하는지 확인하고, Ctrl+C 실험까지 합니다 — 중단돼도 Damp 로 수렴하는가. 이 터미널 로그 캡처가 코드 리뷰의 입장 서류입니다. 여기서 걸러진 문제는 행어 위에서 생기지 않습니다.
- ③ **코드 리뷰(실행 전 설명 가능)** : 실행 전에 멘토에게 코드를 설명합니다 — 아래 7 문항이 기준이며, 통과 서명이 곧 행어 입장권입니다. 5 장 미션 B 에서 답한 그 질문을 이번엔 내 코드에 던집니다.
- ④ **행어 실행** : 6 장 SOP·게이트 순서 그대로, 자기 사본으로 자기 슬롯에서 — 대기 시간의 근거는 시물 실측 + 2~3 초 여유입니다(실기체 상위 명령에는 완료 신호가 없으므로, 6.4 절).

순번	동작 (SEQUENCE 행)	이전 상태	대기 시간	실패 시 복귀
1	예: 손 흔들기(wave)	균형 기립	(초 — 시물 실측+여유)	Damp
2	예: 양팔 들기(hands_up)	균형 기립	(초)	기립 유지 → Damp
3	(추가 동작 — 1·2 완주 확인 후)			

대기 시간 칸이 있는 이유는 5장에서 확인했습니다 — 전이 사이의 대기는 낭비가 아니라, 로봇이 한 상태를 안정적으로 완성할 시간입니다. 대기를 줄이는 실험은 시뮬레이션에서만 합니다.

☐	코드 리뷰 7문항 — 멘토 서명이 행어 입장권
①	SEQUENCE가 설계 시트와 1:1인가 (행 수·순서·대기 시간)
②	모든 동작이 실기체 지원 목록에 있는가 (시물 전용 동작이 남아 있지 않은가)
③	시물 검증 기록이 붙어 있는가 (실행 로그 + Ctrl+C 실험 — ②단계)
④	대기 시간에 근거가 있는가 (시물 실측 + 2~3초 여유)

□	코드 리뷰 7문항 — 멘토 서명이 행어 입장권
⑤	전이 다이어그램(Damp→기립→균형→동작→종료)을 안 보고 그릴 수 있는가
⑥	학생 구역에 보행 명령이 없는가 — 전이·보행은 프레임과 멘토의 소관
⑦	종료 계획(k/d)과 이상 시 첫 행동("손 멈춤·쿨")을 즉답할 수 있는가

7.4 도전 과제 — 보행 + 상체 모션 동시 실행 (시뮬레이션 한정)

여유가 되는 조는 MuJoCo에서 보행과 상체 모션을 동시에 실행하고 균형에 어떤 영향이 있는지 분석합니다 — 배부 저장소의 04_walk_and_wave.py가 이 실험의 출발점이며, 동시 실행을 여는 스위치 (allow_walk_arm)가 코드에 명시되어 있습니다. 팔을 움직이면 무게중심이 이동하고, 이동한 무게중심을 하체가 스텝과 자세 보상으로 따라잡아야 합니다 — 2장에서 배운 "균형의 두 무기"가 동시에 일하는 장면을 처음 보게 되는 것입니다. 이것이 바로 연구실 연구 주제인 loco-manipulation(이동하며 조작하기)의 가장 작은 형태입니다.

- **관찰 포인트** : 팔 동작이 시작되는 순간 스텝 타이밍·보폭이 어떻게 변하는가 (슬로모션으로)
- **비교 실험** : 같은 팔 동작을 정지 상태 / 보행 중에 각각 실행해 상체 흔들림 차이를 기록
- **실기체 반영은 멘토 판단** : 시뮬레이션 결과가 아무리 안정적이어도, 실기체에서의 동시 실행 여부는 멘토가 결정합니다. 학생이 요청할 수 있는 것은 "결과를 보여드리는 것"까지입니다.

실행 조건	정지 상태 + 팔 동작	보행 중 + 팔 동작
상체 흔들림 (슬로모션 관찰)	(기록)	(기록)
스텝 타이밍·보폭 변화	해당 없음	(기록)
완주 여부 (반복 n회 중)	(기록)	(기록)
한 줄 가설	차이가 왜 생겼는가 — 맞든 틀리든 가설이 있어야 다음 실험이 생깁니다 (4장의 원칙 그대로)	

7.5 오후의 시간 배분과 빈출 상황

오후의 리듬 — 시뮬에서 행어로

구간	할 일
14:00 킥오프	통과 기준을 숫자로 확정(학습일지 첫머리) · 분업 확인(과제 1 조 공동 / 과제 2 개인) · "행어에서 디버깅은 계획에 없다" 재확인
14:10 ~ 15:00 과제 1	관찰표 복습 → (필요 시) param_sweep 경계 확인 → SETS 기입·dry-run 낭독 → 본 판정 → 대표 영상 → 비교표 근거·해석 착수. 미통과 시 세트를 한 단계 보수화해

구간	할 일
	재판정 1회(15:00 하드스톱)
15:00 ~ 15:30 과제 2 준비	각자 설계 시트 확정 → 시뮬 선검증 + Ctrl+C 실험 → 개인 사본에 이식 → 코드 리뷰 큐 등록. 같은 시간 멘토는 기체 재가동(precheck → 기립 → watch 재가동 — 기준선은 오전 파일 그대로)
15:30 ~ 17:00 행어 슬롯	리뷰 통과분만, 슬롯표 순서대로 개인 시퀀스 실행(1인 약 18분 + 버퍼) — 종료는 k로 인계, 마지막 슬롯만 d. 슬롯 종료 5분 전 콜
17:00 ~ 17:40 정리	도전 과제(여유 인원) · 비교표·회고 완성 · 조 repo 업로드 · 부록 C의 산출물 목록과 대조
17:40 ~ 18:00 마감	제출물 검수 · HANDOVER.md 3줄 · 기체 종료(스트랩 하중 확인 → Damp → 전원) · 오늘의 한 문장

행어 슬롯표 (화이트보드 양식)

슬롯	시각	실행자(시퀀스 주인)	리뷰 통과 (멘토 서명)	결과 / 비고
1	15:30			
2	15:50			
3	16:10			
4	16:30			

미션 중 빈출 상황 — 막히는 곳은 대개 정해져 있습니다

환경·도구 문제(시작하자마자 무너짐, API 무반응, 토픽 섞임 등)는 부록 B 트러블슈팅을 먼저 봅니다. 아래는 미션 진행 자체에서 자주 나오는 상황입니다.

상황	대응
시뮬레이션에선 됐는데 행어에선 다르게 움직임	고장이 아니라 sim-to-real gap의 체험입니다. 무엇이 어떻게 달랐는지가 회고의 가장 좋은 재료 — 완료 신호 부재·전이 체감 시간·초기 자세 차이부터 의심
시퀀스가 중간에 멈추고 다음 전이가 안 됨 / 팔 액션 거부 (code≠0)	허용되지 않는 상태 전이 순서일 가능성 — 2.4절 상태 기계로 돌아가 현재 모드에서 갈 수 있는 전이인지 확인. 균형 제어 진입 전이거나 직전 동작 직후의 연타가 대부분
두 세트의 반복 실행 결과가 들쭉날쭉함	판정 도구는 매 회 새 시뮬레이터·같은 시작 자세를 자동 보장합니다 — 그런데도 들쭉날쭉하면 반복 로직을 임의 수정했는지, 두 파라미터를 동시에 바꿨는지부터 확인
속달 세트가 안정 세트와 차이가 안 남	안전 경계에서 너무 먼 값을 골랐을 수 있음 — param_sweep으로 경계에 더 다가가되, 경계 밖 값은 세트로 제출하지 않음

상황	대응
판정 도구가 파라미터 적용 오류를 냄	저장소 버전 차이 — 학생이 고치지 말고 멘토에게 (수정 위치가 한 곳으로 정해져 있습니다)
시간 부족	동작 추가·도전 과제를 포기하고 통과 기준 충족과 제출물 완결을 우선. 행어 슬롯을 못 돈 시퀀스는 시물 완주 영상으로 대체 제출 — "작게 완주"가 "크게 미완"보다 낫습니다

이상 징후 시 — 원칙은 하나입니다 비정상 토크, 커지는 IMU 기울기, 이상음 등 이상 징후가 보이면 즉시 손 멈춤 + 콜, 판단은 멘토(비상은 리모컨 댐핑, 스크립트도 스스로 Damp로 수렴합니다). 임의 재시도는 금지입니다. "한 번만 더"가 가장 위험한 말입니다 — 원인 가설 한 줄을 슬롯표에 적은 뒤에만, 멘토 승인으로 재시도합니다.

7.6 제출물과 회고 — 한 주를 남기는 법

제출물 4종 → 조 repo (금요일 오후 내) ① **파라미터 비교표** (자동 생성분에 근거 열한 줄 해석이 채워진 상태로 — 자동 생성분만이면 절반 제출) ② **시퀀스 코드** (개인 사본 + 설계 시트 + 시물 검증 로그 — 코드만으로는 절반 제출) ③ **실행 영상** (과제 1 판정 대표 1회 + 과제 2 행어 실행 — 판정 근거가 되는 장면 위주로) ④ **한 페이지 회고** (아래 3요소)

여기에 오늘 자동으로 쌓인 두 CSV(한 줄 로그·모니터링 기록)를 함께 올립니다 — 주말 분석과 다음 조 기준선의 재료입니다. 개인 산출물(학습일지 월~금, 조사 미션 A·B의 답, 수요일 파라미터 관찰표)은 부록 C대로 개인 repo에 올립니다. 조별 repo에는 Go2 주간과 마찬가지로 다음 조를 위한 인수인계 노트 (HANDOVER.md)를 남깁니다 — ① 기준선 값 요약과 전이 체감 시간·관측 FSM 번호(다르게 나왔다면 반드시) ② 우리를 가장 오래 붙잡은 문제와 해결책 ③ "금요일 아침에 이것부터 확인하세요" 한 줄. 지난주에 받은 인수인계가 고마웠다면, 같은 것을 다음 조에게 돌려줄 차례입니다.

좋은 회고의 3요소

- **사실** : 무엇을 했고 결과가 어땠는가 — 수치와 함께 ("속달 세트 vx 를 올렸더니 3 회 중 1 회 넘어짐, 낙상 4.6 초")
- **해석** : 왜 그랬다고 생각하는가 — 이번 주 배운 개념으로 ("스텝이 무게중심 이탈을 따라잡지 못한 것으로 보임")
- **다음** : 다시 한다면 무엇을 바꾸겠는가 — 다음 플랫폼 주간과 미니프로젝트에서 실제로 써먹을 문장으로

회고에는 sim-to-real 관찰 세 가지를 반드시 포함합니다 — ㉠ 실기체 상위 명령에는 완료 신호가 없어 대기가 시간 기반으로 바뀐 것 ㉡ 전이의 체감 시간이 시물과 달랐던 것 ㉢ 동작 이름이 시물 레지스트리와 실기체 액션으로 매핑되어 옮겨진 것(그중 하나는 실기체 미지원이었던 것). 5.3절 비교 양식의 나머지

행이 바로 이것들입니다.

7.7 평가 관점 (점수가 아니라 성장 기준)

- **안전 절차 준수** — 행어 프로토콜, 게이트·콜 이행, 역할 로테이션
- **실험의 체계성** — 변인 통제, 관찰표·근거 기록, 근거 있는 파라미터 선택
- **sim-to-real 인식** — 시뮬레이션과 실기체의 차이를 어떻게 체감·기술했는가

세 기준 어디에도 "빠른 보행"이나 "화려한 시퀀스"는 없습니다. 평가되는 것은 결과의 크기가 아니라 과정의 질입니다 — 통과 기준을 합의하고, 근거를 남기고, 절차를 지킨 조가 좋은 조입니다. 행어 슬롯을 다 돌았는가보다 전원이 실행자를 경험했는가 먼저입니다.

7.8 자가 점검 — 다음 주로 넘어가기 전에

아래 질문에 스스로 답해 보고, 막히는 항목은 해당 절로 돌아가 확인합니다.

- 과제 1의 두 세트가 각각 무엇을 최적화한 세트인지, 그 차이를 트레이드오프라는 말로 설명할 수 있는가? (7.3 설계 힌트)
- 내 비교표의 모든 주장에 관찰표 행 번호 또는 영상 타임스탬프 근거가 붙어 있는가? — 판정 도구가 채워 주지 않는 두 칸이 무엇인지 아는가? (7.3)
- 내 시퀀스의 상태 전이 그림을 보지 않고 그릴 수 있고, 각 대기 시간의 이유를 말할 수 있는가? (7.3)
- Ctrl+C로 중단했을 때 내 코드가 로봇을 어떤 상태에 남기는지 답할 수 있는가? (7.3, 5.2)
- 설계 시트의 세 열이 시뮬과 실기체로 어떻게 옮겨지는지 — 그리고 실기체 사본에 옮기지 않는 것이 무엇인지 말할 수 있는가? (7.3)
- 과제 1은 시뮬레이션에서, 과제 2는 행어에서 하는 이유 — 무대 배정의 원리를 한 문장으로 말할 수 있는가? (7.1)
- 이상 징후를 발견했을 때의 행동 순서를 즉답할 수 있는가? (7.5)
- 평가 관점 세 가지를 내 조의 오늘 하루에 비추어 스스로 채점할 수 있는가? (7.7)

한 주를 정리하며 — 여러분은 이번 주에 "서 있는 것 자체가 기술"인 로봇을 만났습니다. 균형의 원리를 몸과 화면으로 확인했고, 넘어질 수 있는 실험과 안전한 실험을 무대에 맞게 나누는 법을 배웠고, 같은 설계가 시뮬레이션과 실기체라는 두 무대를 건너가는 것을 직접 옮겨 보았습니다. 다음 플랫폼에서도 사이클은 같습니다 — 조사하고, 시뮬레이션으로 검증하고, 실기체로 옮기기. **무엇이 바뀌어도 이 사이클과 3 역할 체계는 바뀌지 않습니다.**

부록 A. 수요일 퀴즈 대비 체크리스트

✓	스스로 답할 수 있는가
☐	G1의 관절 구성(하체·허리·팔)과 각 부위의 역할
☐	지지 영역과 무게중심으로 "휴머노이드가 넘어지는 이유"를 설명
☐	DDS가 무엇이며 ROS2와 어떤 관계인가
☐	Damp가 모든 절차의 시작이자 복귀 지점인 이유
☐	행어 거치가 기본인 이유 (균형 제어와 연결해 설명)
☐	3장 조사 미션 A의 조사 질문 답 (내 학습일지에 기록되어 있는가)
☐	각 장 말미의 자가 점검 문항 중 막혔던 항목의 답 (2~6장)

부록 B. 트러블슈팅 (G1 주간 빈출)

- **Q. MuJoCo 에서 로봇이 시작하자마자 무너집니다.** A. 제어기(예제 스크립트)가 실행 중인지 확인합니다. 물리만 켜져 있고 제어가 없으면 균형 연산이 없어 당연히 무너집니다 — 2 장의 직관 그대로입니다.
- **Q. 파라미터를 바꿨더니 항상 넘어집니다.** A. 기본값으로 되돌려 정상 보행부터 재현한 뒤, 변경 폭을 절반으로 줄여 다시 시도합니다. "조금씩, 하나씩"이 튜닝의 전부입니다 — 4 장의 관찰표 원칙과 7 장 접근 전략 단계 1:2 가 이것입니다.
- **Q. 실기체에서 API 호출에 반응이 없습니다.** A. 현재 모드가 해당 명령을 받을 수 있는 상태인지(2.4 절 상태 기계·상태 전이 순서), 네트워크 연결이 살아 있는지 멘토와 함께 확인합니다. 임의 재시도 금지 — 특히 기립 관련 명령은 절대 반복 호출하지 않습니다.
- **Q. 다른 조 시뮬레이터의 토픽이 섞여 보입니다.** A. ROS_DOMAIN_ID 를 개인 번호로 설정합니다 (Week 1 FAQ). DDS 자동 발견의 실습실 부작용입니다 — 원리는 3 장에서 설명합니다.
- **미션 진행 중의 빈출 상황**(시물↔행어 거동 차이, 시퀀스 전이 멈춤, 반복 결과 들쭉날쭉 등)은 7.5 절의 대응표를 먼저 봅니다.

부록 C. 이번 주 산출물 정리 (금요일 오후)

- **개인** : 학습일지(월~금) + 조사 미션 A·B 의 답 + 수요일 파라미터 관찰표 → 개인 repo
- **조별** : 파라미터 비교표 + 상체 모션 시퀀스 코드(설계 시트 포함) + 실행 영상 + 한 페이지 회고 + 인수인계 노트(HANDOVER.md, 권장) → 조 repo
- **다음 주 예고** : 다음 플랫폼으로 이동합니다. "조사 → 시뮬레이션 검증 → 실기체"의 사이클과 3 역할 체계는 어디서나 동일합니다.